

CHAP 3. LE FRAMEWORK HIBERNATE

1. Introduction

Hibernate est un outil (framework) de mapping entre les objets et des données stockées dans une base de données relationnelle. Les bases de données supportées sont les principales du marché : DB2, Oracle, MySQL, PostgreSQL, Sybase, SQL Server, Sap DB, Interbase, ...

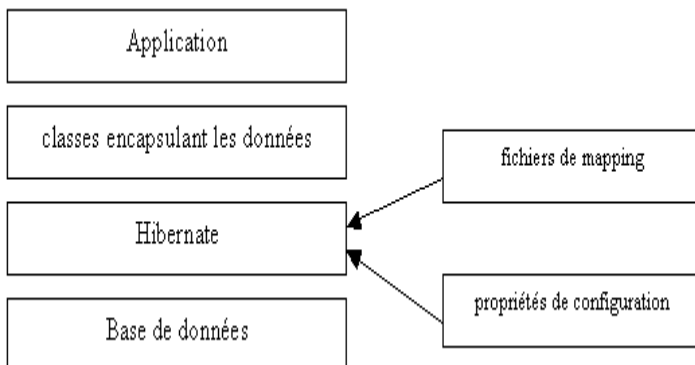
Cette section va utiliser Hibernate avec une base de données de type MySQL possédant une table nommée "personnes".

Field name	Field type
idpersonne *	int(11)
prenompersonne	varchar(50)
nompersonne	varchar(50)
datenaisspersonne	datetime

Hibernate a besoin de plusieurs éléments pour fonctionner :

- une classe de type javabeau qui encapsule les données d'une occurrence d'une table
- un fichier de correspondance qui configure la correspondance entre la classe et la table
- des propriétés de configuration notamment des informations concernant la connexion à la base de données

Une fois ces éléments correctement définis, il est possible d'utiliser Hibernate dans le code des traitements à réaliser. L'architecture d'Hibernate est donc la suivante :



1.1. La création d'une classe qui va encapsuler les données

Cette classe doit respecter le standard des javabeans notamment encapsuler les propriétés dans ses champs private avec des getters et setters et avoir un constructeur par défaut.

Les types utilisables pour les propriétés sont : les types primitifs, les classes String et Dates, les wrappers, et n'importe quelle classe qui encapsule une autre table ou une partie de la table.

Exemple :

```

import java.util.Date;

public class Personnes {

    private Integer idPersonne;
    private String nomPersonne;
    private String prenomPersonne;
    private Date datenaissPersonne;

    public Personnes(Integer idPersonne ,String nomPersonne,
String prenomPersonne, Date datenaissPersonne) {
        this.idPersonne = idPersonne ;
        this.nomPersonne = nomPersonne;
        this.prenomPersonne = prenomPersonne;
        this.datenaissPersonne = datenaissPersonne;
    }

    public Personnes() {
    }

    public Date getDatenaissPersonne() {
        return datenaissPersonne;
    }

    public Integer getIdPersonne() {
        return idPersonne;
    }

    public String getNomPersonne() {
        return nomPersonne;
    }

    public String getPrenomPersonne() {
        return prenomPersonne;
    }

    public void setDatenaissPersonne(Date date) {
        datenaissPersonne = date;
    }

    public void setIdPersonne(Integer integer) {
        idPersonne = integer;
    }

    public void setNomPersonne(String string) {
        nomPersonne = string;
    }

    public void setPrenomPersonne(String string) {
        prenomPersonne = string;
    }

}
  
```

1.2. La création d'un fichier de correspondance

Pour assurer le mapping, Hibernate a besoin d'un fichier de correspondance (mapping file) au format XML qui va contenir des informations sur la correspondance entre la classe définie et la table de la base de données.

Même si cela est possible, il n'est pas recommandé de définir un fichier de mapping pour plusieurs classes. Le plus simple est de définir un fichier de mapping par classe, nommé du nom de la classe suivi par ".**hbm.xml**". Ce fichier doit être situé dans le même répertoire que la classe correspondante ou dans la même archive pour les applications packagées.

Différents éléments sont précisés dans ce document XML :

- la classe qui va encapsuler les données
- l'identifiant dans la base de données et son mode de génération
- le mapping entre les propriétés de classe et les champs de la base de données
- les relations
- ...

Le fichier débute par un prologue et une définition de la DTD utilisée par le fichier XML.

Exemple :

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping
PUBLIC "-//Hibernate/Hibernate Mapping DTD
2.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-
2.0.dtd">
```

Le tag racine du document XML est le tag <hibernate-mapping>. Ce tag peut contenir un ou plusieurs tag <class> : il est cependant préférable de n'utiliser qu'un seul tag <class> et de définir autant de fichiers de correspondance que de classes.

Exemple :

Exemple :

```
<?xml version="1.0"?><!DOCTYPE hibernate-mapping
PUBLIC "-//Hibernate/Hibernate Mapping DTD 2.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-
2.0.dtd"><hibernate-mapping>
<class name="Personnes" table="personnes">
<id name="idPersonne" type="int" column="idpersonne">
<generator class="assigned"/>
</id>
<property name="nomPersonne" column="nompersonne"
type="string" not-null="true" />
<property name="prenomPersonne"
column="prenompersonne" type="string" not-null="true" />
<property name="datenaissPersonne" type="date">
<meta attribute="field-description">date de
naissance</meta>
</property>
</class>
</hibernate-mapping>
```

Le tag <class> permet de préciser des informations sur la classe qui va encapsuler les données.

Ce tag possède plusieurs attributs dont les principaux sont:

Nom	Obligatoire	Rôle
name	oui	nom pleinement qualifié de la classe
table	oui	nom de la table dans la base de données
dynamic-update	non	booléen qui indique de ne mettre à jour que les champs dont la valeur a été modifiée (false par défaut)
dynamic-insert	non	booléen qui indique de ne générer un ordre insert uniquement pour les champs dont la valeur est non nulle (false par défaut)
mutable	non	booléen qui indique si les occurrences peuvent être mises à jour (true par défaut)

Le tag enfant <id> du tag <class> permet de fournir des informations sur l'identifiant d'une occurrence dans la table.

Ce tag possède plusieurs attributs :

Nom	Obligatoire	Rôle
name	non	nom de la propriété dans la classe
type	non	le type Hibernate
column	non	le nom du champ dans la base de données (par défaut le nom de la propriété)

Le tag <generator>, fils obligatoire du tag <id>, permet de préciser quel est le mode de génération d'un nouvel identifiant.

Ce tag possède un attribut :

Attribut	Obligatoire	Rôle
class	oui	précise la classe qui va assurer la génération de la valeur d'un nouvel identifiant. Il existe plusieurs classes fournies en standard par Hibernate qui possèdent un nom utilisable comme valeur de cet attribut.

Les classes de génération fournies en standard par Hibernate possèdent chacun un nom :

Nom	Rôle
increment	incrément d'une valeur dans la JVM

identity	utilisation d'un identifiant auto-incrémenté pour les bases de données qui le supportent (DB2, MySQL, SQL Server, ...)
sequence	utilisation d'une séquence pour les bases de données qui le supportent (Oracle, DB2, PostgreSQL, ...)
native	utilise la meilleure solution proposée par la base de données
assigned	la valeur est fournie par l'application
Etc..	

Certains modes de génération nécessitent des paramètres : dans ce cas, il faut les définir en utilisant un tag fils <param> pour chaque paramètre.

Le tag <property>, fils du tag <class>, permet de fournir des informations sur une propriété et sa correspondance avec un champ dans la base de données.

Ce tag possède plusieurs attributs dont les principaux sont :

Nom	Obligatoire	Rôle
name	oui	précise le nom de la propriété
type	non	précise le type
column	non	précise le nom du champ dans la base de données (par défaut le nom de la propriété)
update	non	précise si le champ est mis à jour lors d'une opération SQL de type update (par défaut true)
insert	non	précise si le champ est mis à jour lors d'une opération SQL de type insert (par défaut true)

Le type doit être soit un type Hibernate (integer, string, date, timestamp, ...), soit les types primitif Java ou de certaines classes de base (int, java.lang.String, float, java.util.Date, ...), soit une classe qui encapsule des données à rendre persistantes.

Le fichier de correspondance peut aussi contenir une description des relations qui existent avec la table dans la base de données.

1.3. Les propriétés de configuration

Pour exécuter Hibernate, il faut lui fournir un certain nombre de propriétés concernant sa configuration pour qu'il puisse se connecter à la base de données.

Ces propriétés peuvent être fournies sous plusieurs formes :

- un fichier de configuration nommé **hibernate.properties** et stocké dans un répertoire inclus dans le classpath
- un fichier de configuration au format XML nommé **hibernate.cfg.xml**
- utiliser la méthode setProperties() de la classe Configuration

Les principales propriétés d'Hibernate pour configurer la connexion JDBC sont :

Nom de la propriété	Rôle
hibernate.connection.driver_class	nom pleinement qualifié de la classe du pilote JDBC
hibernate.connection.url	URL JDBC désignant la base de données
hibernate.connection.username	nom de l'utilisateur pour la connexion
hibernate.connection.password	mot de passe de l'utilisateur
hibernate.connection.pool_size	nombre maximum de connexions dans le pool

Les principales autres propriétés sont :

Nom de la propriété	Rôle
hibernate.dialect	nom de la classe pleinement qualifiée qui assure le dialogue avec la base de données
hibernate.jdbc.use_scrollable_resultset	booléen qui permet le parcours dans les deux sens pour les connexions fournies à Hibernate utilisant des pilotes JDBC 2 supportant cette fonctionnalité
hibernate.show_sql	booléen qui précise si les requêtes SQL générées par Hibernate sont affichées dans la console (particulièrement utile lors du débogage)

Hibernate propose des classes qui héritent de la classe Dialect pour chaque base de données supportée. C'est le nom de la classe correspondant à la base de données utilisée qui doit être obligatoirement fourni à la propriété hibernate.dialect.

Pour définir les propriétés utiles, Voici les méthodes :

Méthode 1 : le plus simple est de définir un fichier de configuration qui en standard doit se nommer **hibernate.properties**. Ce fichier contient des paires clé=valeur pour chaque propriété définie.

Exemple : paramètres pour utiliser une base de données MySQL

```
hibernate.dialect=net.sf.hibernate.dialect.MySQLDialect
hibernate.connection.driver_class=com.mysql.jdbc.Driver
hibernate.connection.url=jdbc:mysql://localhost/testDB
hibernate.connection.username=root
hibernate.connection.password=
```

Le pilote de la base de données utilisée, mysql-connector-java.jar dans l'exemple, doit être ajouté dans le classpath.

Méthode 2 : Il est aussi possible de définir les propriétés dans un fichier au format XML nommé en standard **hibernate.cfg.xml**

Les propriétés sont alors définies par un tag <property>. Le nom de la propriété est fourni grâce à l'attribut « name » et sa valeur est fourni dans le corps du tag.

Exemple :

```
<?xml version='1.0' encoding='utf-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 2.0//EN"
"http://hibernate.sourceforge.net/
hibernate-configuration-2.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.dialect">
net.sf.hibernate.dialect.MySQLDialect
    </property>
    <property name="hibernate.connection.
driver_class">com.mysql.jdbc.Driver</property>
    <property name="hibernate.connection.url">
jdbc:mysql://localhost/testDB</property>
    <property name="hibernate.connection.
username">root</property>
    <property name="hibernate.connection.password">
</property>
    <!-- dialect for MySQL -->
    <!-- <property name="dialect">
net.sf.hibernate.
dialect.MySQLDialect</property>
    <property name="show_sql">
true</property> →
    <property name="dialect">
org.hibernate.dialect.MySQLDialect
```

```
</property>

  <property name="hibernate.show_sql">false
</property>
  <property name="hibernate.transaction.factory_class">
org.hibernate.transaction.JDBCTransactionFactory
</property>
  <property
name="hibernate.hbm2ddl.auto">create</property>
    <!-- Mapping files -->
    <mapping resource="Personnes.hbm.xml"/>
  </session-factory>
</hibernate-configuration>
```

La propriété hibernate.transaction.factory_class contient le nom de classe d'une TransactionFactory qui sera utilisée par l'API Transaction d'Hibernate (la valeur par défaut est JDBCTransactionFactory).

hibernate.hbm2ddl.auto (validate | update | create | create-drop) Valide ou exporte automatiquement le schéma DDL vers la base de données lorsque la SessionFactory est créée au démarrage. La valeur create-drop permet de supprimer le schéma de base de données lorsque la SessionFactory est fermée explicitement.

Nous configurons une SessionFactory d'Hibernate - une fabrique globale responsable d'une base de données particulière. Une SessionFactory peut ouvrir des nouvelles Sessions. Une Session représente une unité de travail simplement "threadée", la SessionFactory est un objet global "thread-safe", instancié une seule fois. Une fois que tous les mappings ont été parsés par la Configuration, l'application doit obtenir une fabrique d'instances de Session. Cette fabrique sera partagée entre tous les threads de l'application. Habituellement, vous voulez que la SessionFactory crée les connexions JDBC et les mette dans un pool pour vous.

Si vous avez plusieurs base de données, utilisez plusieurs configurations <session-factory>, généralement dans des fichiers de configuration différents (pour un démarrage plus facile).

Dans le cadre de l'utilisation au sein d'un serveur d'applications, vous devriez quasiment toujours configurer Hibernate pour qu'il obtienne ses connexions de la DataSource du serveur d'application enregistrée dans le JNDI. Pour cela vous devrez définir au moins une des propriétés suivantes :

Tableau . Propriété d'une Datasource Hibernate

Nom d'une propriété	fonction
hibernate.connection.datasource	Nom JNDI de la datasource
hibernate.jndi.url	URL du fournisseur JNDI (optionnelle)
hibernate.jndi.class	Classe de l'InitialContextFactory du JNDI (optionnelle)

hibernate.connection.username	<i>utilisateur de la base de données (optionnelle)</i>
hibernate.connection.password	<i>mot de passe de la base de données (optionnelle)</i>

Voici un exemple de fichier hibernate.properties pour l'utilisation d'une datasource JNDI fournie par un serveur d'applications :

```
hibernate.connection.datasource =
java:/comp/env/jdbc/test
hibernate.transaction.factory_class =
    org.hibernate.transaction.JTATransactionFactory
hibernate.transaction.manager_lookup_class =
org.hibernate.transaction.JBossTransactionManagerLookup
hibernate.dialect =
org.hibernate.dialect.PostgreSQLDialect
```

Les connexions JDBC obtenues à partir d'une datasource JNDI participeront automatiquement aux transactions gérées par le conteneur du serveur d'applications.

Configuration de la stratégie transactionnelle

L'API de la Session Hibernate est indépendante de tout système de démarcation des transactions qui peut être présent dans votre architecture. Si vous laissez Hibernate utiliser l'API JDBC directement via un pool de connexion, vous devrez commencer et terminer vos transactions en utilisant l'API JDBC. Si votre application tourne à l'intérieur d'un serveur d'application J2EE, vous voudrez peut être utiliser les transactions gérées par les beans (BMT) et appeler l'API JTA et UserTransaction lorsque cela est nécessaire.

Pour conserver votre code portable entre ces deux environnements (et d'autres éventuels) nous vous recommandons d'utiliser l'API optionnelle Transaction d'Hibernate, qui va encapsuler et masquer le système de transaction sous-jacent. Pour cela, vous devez préciser une classe de fabrique d'instances de Transaction en positionnant la propriété hibernate.transaction.factory_class.

Il existe trois choix standards (fournis) :

1) org.hibernate.transaction.JDBCTransactionFactory

délègue aux transactions de la base de données (JDBC). Valeur par défaut.

2)org.hibernate.transaction.JTATransactionFactory

délègue à CMT si une transaction existante est sous ce contexte (ex: méthode d'un EJB session), sinon une

nouvelle transaction est entamée et une transaction gérée par le bean est utilisée.

3)org.hibernate.transaction.CMTTransactionFactory

délègue à aux transactions JTA gérées par le conteneur

Certaines fonctionnalités d'Hibernate (i.e. le cache de second niveau, l'association automatique des Session à JTA, etc.) nécessitent l'accès au TransactionManager JTA dans un environnement "managé". Dans un serveur d'application, vous devez indiquer comment Hibernate peut obtenir une référence vers le TransactionManager, car J2EE ne fournit pas un seul mécanisme standard.

Tableau 3.10. TransactionManagers JTA

Fabrique de Transaction
org.hibernate.transaction.JBossTransactionManagerLookup
org.hibernate.transaction.WeblogicTransactionManagerLookup
org.hibernate.transaction.WebSphereTransactionManagerLookup
org.hibernate.transaction.WebSphereExtendedJTATransactionL
org.hibernate.transaction.OrionTransactionManagerLookup
org.hibernate.transaction.ResinTransactionManagerLookup
org.hibernate.transaction.JOTMTTransactionManagerLookup
org.hibernate.transaction.JOnASTransactionManagerLookup
org.hibernate.transaction.JRun4TransactionManagerLookup
org.hibernate.transaction.BESTransactionManagerLookup

Démarcation des transactions

Une application utilisant Hibernate peut s'exécuter dans un environnement léger n'offrant pas la gestion automatique des transactions (application autonome, application web simple ou applications Swing) ou dans un environnement J2EE offrant des services de gestion automatique des transactions JTA.

Dans un environnement simple, Hibernate a généralement la responsabilité de la gestion de son propre pool de connexions à la base de données. Le développeur de l'application doit manuellement délimiter les transactions. En d'autres mots, il appartient au développeur de gérer les appels à Transaction.begin() , Transaction.commit() et Transaction.rollback() .

Un environnement transactionnel J2EE (serveur d'application J2EE) doit offrir la gestion des transactions au niveau du container J2EE. Les bornes de transaction peuvent normalement être définies de manière déclarative dans les descripteurs de déploiement d'EJB Session, par exemple. La gestion programmatique des transactions n'y est donc pas nécessaire. Même

les appels à `Session.flush()` sont faits automatiquement.

Environnement non managé

Si la couche de persistance Hibernate s'exécute dans un environnement non managé, les connexions à la base de données seront généralement prises en charge par le mécanisme de pool d'Hibernate. La gestion de la session et de la transaction se fera donc de la manière suivante:

```
// Non-managed environment idiom
Session sess = factory.openSession();
Transaction tx = null;
try {
    tx = sess.beginTransaction();

    // do some work
    ...

    tx.commit();
}
catch (RuntimeException e) {
    if (tx != null) tx.rollback();
    throw e; // or display error message
}
finally {
    sess.close();
}
```

Vous n'avez pas à invoquer `flush()` explicitement sur la Session - l'appel de `commit()` déclenchera automatiquement la synchronisation. Un appel à `close()` marque la fin de la session. La conséquence directe est que la connexion à la base de données sera relâchée par la session. Ce code est portable et fonctionne dans les environnements non managé ET les environnements JTA.

Une solution plus flexible est la gestion par contexte fourni par Hibernate que nous avons déjà rencontré:

```
// Non-managed environment idiom with
getCurrentSession()
try {
    factory.getCurrentSession().beginTransaction();

    // do some work
    ...

    factory.getCurrentSession().getTransaction().commit()
;
}
catch (RuntimeException e) {
    factory.getCurrentSession().getTransaction().rollback(
);
    throw e; // or display error message
}
```

Utilisation de JTA

Si votre couche de persistance s'exécute dans un serveur d'application (par exemple, derrière un EJB Session Bean), toutes les datasources utilisées par Hibernate feront automatiquement partie de transactions JTA globales. Hibernate propose deux stratégies pour réussir cette intégration.

Si vous utilisez des transactions gérées par un EJB (bean managed transactions - BMT), Hibernate informera le serveur d'application du début et de la fin des transactions si vous utilisez l'API Transaction. Ainsi, le code de gestion des transactions sera identique dans les deux types d'environnements.

```
// BMT idiom
Session sess = factory.openSession();
Transaction tx = null;
try {
    tx = sess.beginTransaction();

    // do some work
    ...

    tx.commit();
}
catch (RuntimeException e) {
    if (tx != null) tx.rollback();
    throw e; // or display error message
}
finally {
    sess.close();
}
```

Ou encore, avec la gestion automatique de contexte:

```
// BMT idiom with getCurrentSession()
try {
    factory.getCurrentSession().beginTransaction();

    // do some work
    ...

    factory.getCurrentSession().getTransaction().commit()
;
}
catch (RuntimeException e) {
    factory.getCurrentSession().getTransaction().rollback(
);
    throw e; // or display error message
}
```

Avec CMT, la démarcation des transactions est faite dans les descripteurs de déploiement des Beans Sessions et non de manière programmatique, ceci réduit le code:

```
// CMT idiom
Session sess = factory.getCurrentSession();

// do some work
...
```

Dans un EJB CMT même le rollback intervient automatiquement, puisqu'une RuntimeException non traitée et soulevée par une méthode d'un bean session indique au conteneur d'annuler la transaction globale. *Ceci veut donc dire que vous n'avez pas à utiliser l'API Transaction d'Hibernate dans CMT.*

Notez que le fichier de configuration Hibernate devrait contenir les valeurs
org.hibernate.transaction.JTATransactionFactory dans un environnement BMT ou
org.hibernate.transaction.CMTTransactionFactory dans un environnement CMT là où vous configurez votre transaction factory Hibernate

1.4. L'utilisation d'Hibernate

Pour utiliser Hibernate dans le code, il est nécessaire de réaliser plusieurs opérations :

- création d'une instance de la classe Configuration
- création d'une instance de la classe SessionFactory (Un cache immuable des mappings vers une (et une seule) base de données.)
- création d'une instance de la classe Session qui va permettre d'utiliser les services d'Hibernate

Si les propriétés sont définies dans le fichier hibernate.properties, il faut tout d'abord créer une instance de la classe Configuration. Pour lui associer la ou les classes encapsulant les données, la classe propose deux méthodes :

- addFile() ou addResource qui attend en paramètre le nom du fichier de mapping
- addClass() qui attend en paramètre un objet de type Class encapsulant la classe. Dans ce cas, la méthode va rechercher un fichier nommé nom_de_la_classe.hbm.xml dans le classpath (ce fichier doit se situer dans le même répertoire que le fichier .class de la classe correspondante)

Une instance de la classe Session est obtenue à partir d'une fabrique de type SessionFactory. Cet objet est obtenu à partir de l'instance du type Configuration en utilisant la méthode buildSessionFactory() et représente une conversation entre l'application et l'entrepôt de persistance.

La méthode openSession() de la classe SessionFactory permet d'obtenir une instance de la classe Session.

Par défaut, c'est la méthode openSession() qui va ouvrir une connexion vers la base de données en utilisant les informations fournies par les propriétés de configuration.

Exemple :

```
//import net.sf.hibernate.*;
```

```
// import net.sf.hibernate.cfg.Configuration;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
import java.util.Date;

public class TestHibernate1 {

    public static void main(String args[]) throws Exception {
        Configuration config = new Configuration();
        config.addClass(Personnes.class);
        SessionFactory sessionFactory =
            config.buildSessionFactory();
        Session session = sessionFactory.openSession();

        ...
    }
}
```

OU BIEN

```
Configuration config = new Configuration();
config.addResource("Client.hbm.xml");
// Pour Mysql

config.setProperty("hibernate.connection.url",
    "jdbc:mysql://localhost/Base");
config.setProperty("hibernate.connection.driver_class", "com.mysql.jdbc.Driver");
config.setProperty("hibernate.connection.username", "root");

config.setProperty("hibernate.connection.password", "");
// dialect for MySQL -->
config.setProperty("hibernate.dialect", "org.hibernate.dialect.MySQLDialect");
config.setProperty("hibernate.show_sql", "false");
config.setProperty("hibernate.connection.pool_size", "10");

// Pour PosGresql
config.setProperty("hibernate.connection.url", "jdbc:postgresql://localhost:5432/base");

config.setProperty("hibernate.connection.driver_class", "org.postgresql.Driver");

config.setProperty("hibernate.connection.username", "postgres");

config.setProperty("hibernate.connection.password", "");
config.setProperty("hibernate.dialect", "org.hibernate.dialect.PostgreSQLDialect");
config.setProperty("hibernate.transaction.factory_class", "org.hibernate.transaction.JDBCTransactionFactory");

SessionFactory sessionFactory
    =config.buildSessionFactory();
    session =sessionFactory.openSession();
```

Pour une utilisation du fichier **hibernate.cfg.xml**, il faut créer une occurrence de la classe Configuration, appeler sa méthode **configure()** qui va lire le fichier XML et appeler la méthode buildSessionFactory() de l'objet renvoyé par la méthode configure().

Exemple : 7

```
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class TestHibernate1 {
    public static void main(String args[]) throws Exception {
        SessionFactory sessionFactory = new
        Configuration().configure().buildSessionFactory();
        Session session = sessionFactory.openSession();
        ...
    }
}
```

Il est important de clôturer l'objet Session, une fois que celui-ci est devenu inutile, en utilisant la méthode `close()`.

Remarque :

A partir de Hibernate 4.x, la classe `AnnotationConfiguration` est dépréciée et remplacée par :

```
; // pour Hibernate version 4.x
import org.hibernate.service.ServiceRegistry;
import org.hibernate.service.ServiceRegistryBuilder
```

```
// pour Hibernate 5.X
import org.hibernate.service.ServiceRegistry;
import org.hibernate.boot.registry.
StandardServiceRegistryBuilder;
```

```
/*      VERSION 4
        // loads configuration and mappings
```

```
    Configuration configuration = new
    Configuration().configure();
    ServiceRegistryBuilder registry = new
    ServiceRegistryBuilder();
    registry.applySettings(configuration.getProperties());
    ServiceRegistry serviceRegistry =
    registry.buildServiceRegistry();
    SessionFactory sessionFactory =
    configuration.buildSessionFactory(serviceRegistry);
```

```
*
```

```
//      Version 5
    Configuration configuration = new
    Configuration().configure();
    ServiceRegistry serviceRegistry= new
    StandardServiceRegistryBuilder()
    applySettings (configuration.getProperties()).build();
    // builds a session factory from the service registry
    SessionFactory sessionFactory =
    configuration.buildSessionFactory(serviceRegistry);
    Session session = sessionFactory.openSession();
    .....
```

États des objets *mappés* de l'application Java

Ces trois états ne concernent que les instances de classes *mappées*.

persistent

(persistant): cet état n'est possible que quand une session est ouverte, il correspond au fait qu'Hibernate s'occupe complètement de la persistance de l'objet. L'état de l'objet devient détaché dès que la session est fermée.

detached

(détaché): précédemment persistant, mais actuellement non associé à une Session, car la session a été fermée (`close()`).

transient

(éphémère): associé avec aucune Session, c'est l'état d'un objet qui vient juste d'être créé par `new`, que ce soit en dehors ou dans une session.

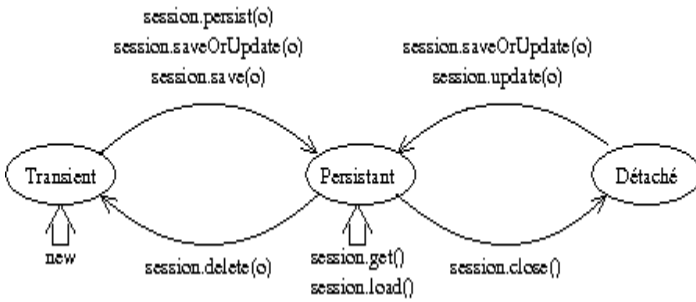
Seul un objet dans l'état persistant peut faire l'objet d'une mise à jour automatique dans la base de données lors d'un *flush* sur la session.

Un objet ne peut être dans l'état persistant que pendant qu'une session est ouverte. Dès que la session sera fermée, cet objet passera dans l'état détaché.

Les changements d'état d'une instance de classe *mappée* se font aussi au sein d'une session :

- un objet instancié directement avec `new` est *transient* : il n'est associé à aucune session. Pendant une session il devient *persistent* suite aux opérations : `session.save(obj)`, `session.persist(obj)` ou `session.saveOrUpdate(obj)`.
- un nouvel objet obtenu, pendant une session, depuis la base de données par `session.get()` ou `session.load()` est *persistent*. Il devient *transient* avec `session.delete(obj)`. Il devient *detached* lors de la fermeture de la session (`session.close()`)
- Un ancien objet *persistent* pendant une session précédente, est initialement *detached* lors d'une nouvelle session. Il devient *persistent* avec `session.update(obj)`, `session.saveOrUpdate(obj)`, `session.lock(obj)` ou comme nouvelle instance persistante avec `session.merge(obj)`.

Figure 1: Cette figure résume une partie des transitions d'états possibles pendant une session. Les deux seuls états initiaux possibles (transient et persistant) sont indiqués par ↑. Remarquez qu'il n'y a que des méthodes de Session, la session devant être ouverte.



Les objets persistants modifiés sont détectés lors d'un `flush()` de la `Session` et des ordres SQL (**insert**, **update** ou **delete**) sont alors exécutés pour garantir leur persistance.

Création d'une classe `HibernateUtil`

Cette classe permet d'obtenir facilement une session Hibernate.

```

package exa;
import org.hibernate.*;
import org.hibernate.cfg.*;
public class HibernateUtil {
    private static final SessionFactory sessionFactory;
    static {
        try { // Crée la SessionFactory
            sessionFactory = new
Configuration().configure().buildSessionFactory();
        } catch (HibernateException ex) {
            throw new RuntimeException("Pb de conf : " +
ex.getMessage(), ex);
        }
    }
    public static final ThreadLocal session = new
ThreadLocal();
    //obtention de la session courante
    public static Session currentSession() throws
HibernateException {
        Session s = (Session) session.get();
        // Ouvre une nouvelle Session, si ce Thread n'en a
aucune
        if (s == null) { s = sessionFactory.openSession();
session.set(s); }
        return s;
    }
    //fermeture de session
    public static void closeSession() throws
HibernateException {
        Session s = (Session) session.get();
        session.set(null);
        if (s != null)
            s.close();
    }
}
  
```

1.5. La persistance d'une nouvelle occurrence

Pour créer une nouvelle occurrence dans la source de données, il suffit de créer une nouvelle instance de la classe encapsulant les données, de valoriser ses propriétés et d'appeler la méthode `save()` de la session

en lui passant en paramètre l'objet encapsulant les données.

La méthode `save()` n'a aucune action directe sur la base de données. Pour enregistrer les données dans la base, il faut réaliser un commit sur la connexion ou la transaction ou faire appel à la méthode `flush()` de la classe `Session`.

`save()` va s'occuper de sauvegarder un objet qui n'est pas encore présent dans la base de données.

On peut utiliser `saveOrUpdate()`, quand à elle, va faire une vérification sur l'état de l'objet et en fonction de son état (transient, détaché ou persistant), il va effectuer l'action correcte. Vous n'avez ainsi plus de problèmes pour savoir l'état de votre objet

Exemple :

```

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import test.hibernate.*;
import java.util.Date;
import test.hibernate.Personnes;
public class TestHibernate1 {

    public static void main(String args[]) throws Exception {
        SessionFactory sessionFactory = new
Configuration().configure().buildSessionFactory();
        Session session = sessionFactory.openSession();
        Transaction tx = null;
        try {
            tx = session.beginTransaction();
            Personnes personne = new Personnes(new
Integer(10),"nom3", "prenom3", new Date());
            /* Personnes personne = new Personnes();
            personne.setIdPersonne(new Integer(10)) ;
            personne.setNomPersonne("RABE") {
            personne.setPrenomPersonne("Deepak");
            personne.setDatednaissPersonne(new Date()) ;
            */
            session.save(personne);
            session.flush() ;
            tx.commit();
        } catch (Exception e) {
            if (tx != null) {
                tx.rollback();
            }
            throw e;
        } finally {
            session.close();
        }

        sessionFactory.close();
    }
}
  
```

//save exemple – sans transaction

```
Session session = sessionFactory.openSession();
```

```

Personnes pers = new Personnes(
new Integer(10),"nom1", "prenom1", new Date());
long id = (Long) session.save(personne);
System.out.println("Employee save called without
transaction, id="+id);
session.flush();

```

On peut aussi persister un objet avec la méthode persist similaire à save mais dans une transaction.

```

Session session2 = sessionFactory.openSession();
Transaction tx2 = session2.beginTransaction();
Personnes pers = new Personnes(
new Integer(10),"nom1", "prenom1", new Date());
session2.persist(pers);
emp2.setName("Kumar"); // sera modifié dans bd
System.out.println(" Name updated");
tx2.commit();

```

Problème de sauvegarde d'un même objet dans 2 sessions différents

```

public class SaveEntityAgainInAnotherSession
{ public static void main(String[] args)
{
    Session sessionOne =
    HibernateUtil.getSessionFactory().openSession();
    sessionOne.beginTransaction();

    //Create new Employee object
    EmployeeEntity emp = new EmployeeEntity();
    emp.setEmployeeId(1);
    emp.setFirstName("Lokesh");

    //Save employee
    sessionOne.save(emp);
    sessionOne.getTransaction().commit();
    Session sessionTwo =
    HibernateUtil.getSessionFactory().openSession();
    sessionTwo.beginTransaction();
    emp.setLastName("temp");
    //Save employee again second time
    sessionTwo.save(emp); // duplication
d'identifiant
    sessionTwo.getTransaction().commit();
    HibernateUtil.shutdown();
}
}

```

Hibernate: insert into Employee (FIRST_NAME, LAST_NAME, ID) values (?, ?, ?)ERROR
 SQLExceptionHelper:146 - Violation of unique constraint SYS_PK_49: **duplicate value(s)** for column(s) ID in statement [insert into Employee (FIRST_NAME, LAST_NAME, ID) values (?, ?, ?)]

Note: La seconde save() cause la duplication de ligne dans des sessions différentes, mais dans une même session, les deux méthodes save fonctionneront correctement comme le montre l'exemple suivant.

```

public class SaveEntityAgainInSameSession
{ public static void main(String[] args)
{
    Session sessionOne = .....
    sessionOne.beginTransaction();
    //Create new Employee object

```

```

EmployeeEntity emp = new EmployeeEntity();
emp.setEmployeeId(1);
emp.setFirstName("Lokesh");
sessionOne.save(emp); //Save employee
emp.setLastName("temp");
//Save employee again second time
sessionOne.save(emp);
sessionOne.getTransaction().commit();
}
}

```

Hibernate: insert into Employee (FIRST_NAME, LAST_NAME, ID) values (?, ?, ?)Hibernate: update Employee set FIRST_NAME=?, LAST_NAME=? where ID=?

Exemple2: sauve automatique de modification

```

Session sessionOne =
HibernateUtil.getSessionFactory().openSession();
sessionOne.beginTransaction();

//Create new Employee object
Employee emp = new Employee();
emp.setEmployeeId(1); emp.setFirstName("Lokesh")
;
emp.setLastName("Gupta");
sessionOne.save(emp); //Save employee
emp.setLastName("temp"); // update saved
automatically
sessionOne.getTransaction().commit();

Session sessionTwo =
HibernateUtil.getSessionFactory().openSession();
sessionTwo.beginTransaction();

EmployeeEntity employee = ( EmployeeEntity )
sessionTwo.load(EmployeeEntity.class, 1);
System.out.println(employee.getLastName());
sessionTwo.getTransaction().commit();

```

Solution : utilisation de la méthode SaveOrUpdate pour sauver une seconde fois

```

Session sessionOne =
HibernateUtil.getSessionFactory().openSession();
sessionOne.beginTransaction();

//Create new Employee object
EmployeeEntity emp = new EmployeeEntity();
emp.setEmployeeId(1);
emp.setFirstName("Lokesh");
emp.setLastName("Gupta");
sessionOne.save(emp); //Save employee
sessionOne.getTransaction().commit();

Session sessionTwo =
HibernateUtil.getSessionFactory().openSession();
sessionTwo.beginTransaction();
emp.setLastName("temp");
//Save employee again second time
sessionTwo.saveOrUpdate(emp);
sessionTwo.getTransaction().commit();

```

1.6. L'obtention d'une occurrence à partir de son identifiant

La méthode **load()** de la classe Session permet d'obtenir une instance de la classe des données encapsulant les données de l'occurrence de la base dont l'identifiant est fourni en paramètre.

Il existe deux surcharges de la méthode :

- la première attend en premier paramètre le type de la classe des données et renvoie une nouvelle instance de cette classe
- la seconde attend en paramètre une instance de la classe des données et la met à jour avec les données retrouvées

Exemple :

```
.....

public class TestHibernate2 {

    public static void main(String args[]) throws Exception {
        SessionFactory sessionFactory = new
        Configuration().configure().buildSessionFactory();
        Session session = sessionFactory.openSession();

        try {
            Personnes personne = (Personnes)
            session.load(Personnes.class, new Integer(3));
            System.out.println("nom = " +
            personne.getNomPersonne());
        } finally {
            session.close();
        }
        sessionFactory.close();
    }
}
```

1.7. La mise à jour d'une occurrence

Pour mettre à jour une occurrence dans la source de données, il suffit d'appeler la méthode update() de la session en lui passant en paramètre l'objet encapsulant les données.

La méthode saveOrUpdate() laisse Hibernate choisir entre l'utilisation de la méthode save() ou update() en fonction de la valeur de l'identifiant dans la classe encapsulant les données.

Exemple :

```
/// import.....
public class TestHibernate3 {

    public static void main(String args[]) throws Exception {
        SessionFactory sessionFactory = new
        Configuration().configure().buildSessionFactory();
        Session session = sessionFactory.openSession();
        Transaction tx = null;
```

```
try {
    tx = session.beginTransaction();
    Personnes personne =
    (Personnes)session.get(Personnes.class, new Integer(10));
    personne.setldPersonne(new Integer(12));
    personne.setNomPersonne("RIVO") {
    personne.setPrenomPersonne("Jean");
    sess.update(session);
    tx.commit();
} catch (Exception e) {
    tx.rollback();

} finally {
    session.close();
}

sessionFactory.close();
}
```

Modifier des objets persistants

La manière la plus directe de mettre à jour l'état d'un objet est de le charger avec load(), et puis le manipuler directement, tant que la Session est ouverte sans besoin de transaction:

```
Employe emp = (Employe)
    session.load( Employe.class, new Long(69) );
```

```
emp.setName("PK");
```

```
session.flush(); // les changements sur Employe sont
    automatiquement détectés et persistés
```

Modifier des objets détachés

Beaucoup d'applications ont besoin de récupérer un objet dans une transaction, l'envoyer à la couche interfacée avec l'utilisateur pour les manipulations, puis sauvegarder les changements dans une nouvelle transaction. Hibernate supporte ce modèle en permettant pour le réattachement d'instances détachées l'utilisation des méthodes Session.update ou session.saveOrUpdate() ou Session.merge()

Exemple 1: utilisation de update pour modifier des objets dans une autre session

```
Session session1 = sessionFactory.openSession();
session1.beginTransaction();
//recupérer données de la bd
Employe emp = (Employe) session1.get(Employe.class,
2);
System.out.println("UserName : " +
emp.getUserName());
session1.getTransaction().commit();
session1.close(); // emp détaché

Session session2 =
sessionFactory.openSession();
session2.beginTransaction();
```

```
emp.setUsername("Update Name"); //Update Record
//Update the data into the database
    session2.update(emp);
    session2.getTransaction().commit();
    session2.close();
```

Exemple 2: utilisation de la méthode merge

```
Session session1 = sessionFactory.openSession();
Transaction tx = session1.beginTransaction();
Item item1 = (Item) session1.get(Item.class, new
    Long(12));
tx.commit();
session1.close(); // end of first session, item1 is
    detached
item1.getId(); // The database identity is "12"
item1.setDescription("my new description");

Session session2 = sessionFactory.openSession();
Transaction tx2 = session2.beginTransaction();
Item item2 = (Item) session2.get(Item.class, new
    Long(12));
//trying to reattach the first object in 2nd session
session2.update(item1); //Throws
    NonUniqueObjectException (présence de 2 objets
    de même identifiant)
tx2.commit();session2.close();
```

Solution : fusionner l'objet détaché item1 avec l'objet persistant item2

```
Session session = sessionFactory1.openSession();
Transaction tx = session.beginTransaction();
Item item1 = (Item) session.get(Item.class, new
    Long(12));
tx.commit();
session.close(); // item is detached but in cache

item1.getId(); // The database identity is "12"
item1.setDescription("my new description");

Session session2 = sessionFactory.openSession();
Transaction tx2 = session2.beginTransaction();
Item item2 = (Item) session2.get(Item.class, new
    Long(12));
Item mergedItem = (Item) session2.merge(item); //
    Success!
//on recherche dans le cache de persistance, trouve
    entity2 (item2) et le fusionne ( merge) avec
    entity1 (item1) et retourne entity2
System.out.println (mergedItem == item1); // false
System.out.println (mergedItem == item2); // true :item1
    est encore détachée
tx2.commit();
session2.close();
```

Hibernate ne peut pas modifier les entités détachées. A sa place, il recherche l'entité ayant le même identifiant que l'entité détachée dans le cache de persistance de session. En trouvant l'entité (entity2) , il fusionne les données de entity1 dans entity2 et retourne la référence à l'objet persistant existant (entity2) .

L'objet entity1 est encore détachée mais les changements sont fusionnés dans l'entité entity2 -

Modifier une entité détachée entity1 par fusion sans avoir entity2 en cache

```
public static void mergeCase2() {
    Session session = sessionFactory.openSession();
    Entity entity1 = (Entity) session.get(Entity.class, 1);
    String data = entity1.getData();
    session.close();
    System.out.println("data is " + data);
    entity1.setData("mergeCase2");

    Session newSession =
        sessionFactory.openSession();
    Transaction transaction =
        newSession.beginTransaction();
    Entity mergedEntity = (Entity)
        newSession.merge(entity1);
    //it searches in persistence cache, not found
    -searches db - fetches dbentity, merges it with
        entity1 and returns dbentity
    System.out.println((mergedEntity == entity1)); // false
    transaction.commit(); //will cause an update to occur
        here
    newSession.close();
}
```

Utilisation de la méthode merge pour insérer un nouvel objet dans le cas où l'objet n'existe pas dans la bd

```
public static void mergeCase3() {
    Entity entity1=new Entity();//new object created;in
        Transient state
    entity1.setData("mergeCase3");
    System.out.println("Creating a transient object ");

    Session newSession = sessionFactory.openSession();
    Transaction transaction =
        newSession.beginTransaction();
    Entity mergedEntity = (Entity)
        newSession.merge(entity1);
    //it searches in pc, will find no match, no match in db
        either
    // creates a new instance and schedules it for
        insertion,
    //new entity merges it with entity1 and returns new entity
    System.out.println( (mergedEntity == entity1)); // false
    transaction.commit(); //will cause an insert to occur
        here
    newSession.close();
}
```

1.8. La suppression d'une ou plusieurs occurrences

La méthode **delete()** de la classe Session permet de supprimer une ou plusieurs occurrences en fonction de la version surchargée de la méthode utilisée.

Pour supprimer une occurrence encapsulée dans une classe, il suffit d'invoquer la classe en lui passant en paramètre l'instance de la classe.

Pour supprimer plusieurs occurrences, voire toutes, il faut passer en paramètre de la méthode delete(), une chaîne de caractères contenant la **requête HQL** pour préciser les éléments concernés par la suppression.

Exemple : suppression de toutes les occurrences de la table

```
public void deletePersonnes1(
Integer idPersonne)
{
    SessionFactory sessionFactory = new
Configuration().configure().buildSessionFactory();
    Session session = sessionFactory.openSession();
    Transaction transaction = null;
    try {

        transaction = session.beginTransaction();
        Personnes personne = (Personnes)session.get(
Personnes.class,idPersonne);
        session.delete(personne);
        // session.delete("from Personnes");// pour
supprimer toutes les occurrences
        transaction.commit();
    } catch (HibernateException e) {
        transaction.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
}

public void deletePersonnes2(Integer idPersonne)
{
    Session sess = null;
    try {
        SessionFactory sessionFactory = new
Configuration().configure().buildSessionFactory();
        Session session = sessionFactory.openSession();
        String hql = "delete from Personnes p where
p.idPersonne="+idPersonne;
        Query query = session.createQuery(hql);
        int row = query.executeUpdate();
        if (row == 0){
            System.out.println("Doesn't deleted any row!");
        }
        else{
            System.out.println("Deleted Row: " + row);
        }
        session.close();
    }
    catch(Exception e){
        System.out.println(e.getMessage());
    }
}
```

2. INTERROGATION D'UNE BASE DE DONNEES AVEC HIBERNATE

- Le langage de requête HQL
- L'API Criteria
- Hibernate native SQL

2. 1. Le langage de requête HQL

Hibernate Query Language est un langage propre d'interrogation des classes persistantes avec une syntaxe proche du SQL, et commun à toutes les base de données. Cependant les objets manipulés sont les classes et membres du mapping contrairement au SQL qui réalise les requêtes directement sur la base de données. Aussi, le HQL contient des fonctionnalités spécifiques au modèle objet. Les requêtes HQL peuvent être formées des éléments principaux suivants :

- les clauses (from, select, where, order by, ...)
- les fonctions d'agrégation (count, sum, avg, max, ...)
- les sous-requêtes (il s'agit de requêtes HQL dont le résultat est utilisé dans une requête HQL principale.

Il est également possible de définir des requêtes utilisant des paramètres nommés grâce à un objet implémentant l'interface **Query**. Un objet de type Query est obtenu en invoquant la méthode **createQuery()** de la classe Session avec comme paramètre la requête HQL.

Pour parcourir la collection des occurrences trouvées, l'interface Query propose la méthode list() qui renvoie une collection de type List ou la méthode iterate() qui renvoie un itérateur sur la collection.

Exemple : rechercher toutes les occurrences d'une table

```
public void listPersonnes()
{
    Session session = .....;
    Transaction transaction = null;
    try {
        transaction = session.beginTransaction();

        /* List personnes = session.find
("from Personnes"); // version 2

        List personnes = session.find
("from Personnes p order by
p.nomPersonne desc");

        for (int i = 0; i < personnes.size(); i++) {
            Personnes personne = (Personnes) personnes.get(i);
            System.out.println("nom = " +
personne.getNomPersonne());
        }
    }
}
```

```

*/
/*      List liste = session.createQuery("from
Personnes").
list();
    for (Iterator it = liste.iterator(); iterator.hasNext();)
    {
        Personne pers = (Personne) it.next();
        System.out.println(pers.
getNomPersonne());
    }
*/
String SQL_QUERY =
"from Personnes p";
Query query = session.createQuery(SQL_QUERY);
for(Iterator it=query.iterate();it.hasNext();)
{
    Personnes pers=(Personnes)it.next();
    System.out.println(pers.getNomPersonne());
}

    transaction.commit();
} catch (HibernateException e) {
    transaction.rollback();
    e.printStackTrace();
} finally {
    session.close();
}
}

```

La première surcharge permet de fournir un seul paramètre : elle attend en paramètre la requête, la valeur du paramètre et le type du paramètre. Dans la requête de l'exemple suivant, un alias nommé « **p** » ou **as p** est défini pour la classe Personnes. Le mode de fonctionnement d'un alias est similaire en HQL et en SQL.

Plusieurs classes peuvent apparaître, ce qui conduira à un produit cartésien (encore appelé jointures croisées).

from Formula, Parameter
from Formula as form, Parameter as param

Exemple :

```

public void (String[] args) {
    Session session = null;

    try{
        // This step will read hibernate.cfg.xml
        and prepare hibernate for use
        SessionFactory sessionFactory = new
        Configuration().configure().buildSessionFactory();
        Session session =sessionFactory.openSession();
        //Query using Hibernate Query Language

        String SQL_QUERY = "from Personnes p where

```

```

p.nomPersonne='"+nomPers+"'";
    Query query =
session.createQuery(SQL_QUERY);
    for(Iterator it=query.iterate();it.hasNext();){
        Personnes pers=(Personnes)it.next();
        System.out.println(pers.getNomPersonne());
        System.out.println(pers.getPrenomPersonne());

    }

    //Where Clause With Select Clause
    SQL_QUERY ="Select p.IdPersonne, p.Nom from
Personnes p order by p.nomPersonne desc where
p.IdPersonne='"+ "+nomPers+"'";
    query = session.createQuery(SQL_QUERY);
    for(Iterator it=query.iterate();it.hasNext();){
        Object[] row = (Object[]) it.next();
        System.out.println("IdPersonne: " + row[0]);
        System.out.println("Name: " + row[1]);

    }

    session.close();
}catch(Exception e){
    System.out.println(e.getMessage());
}finally{
}
}

```

Dans la requête du précédent exemple, un alias nommé « **p** » ou **as p** est défini pour la classe Personnes. Le mode de fonctionnement d'un alias est similaire en HQL et en SQL.

Il est aussi possible d'utiliser la clause « order by » dans une requête HQL pour définir l'ordre de tri des occurrences et des fonctions telles que count() pour compter le nombre d'occurrences etc...

Exemple :

```

public class TestHibernate5 {

    public static void main(String args[]) throws Exception {
        int count = 0;
        SessionFactory sessionFactory = new
        Configuration().configure().buildSessionFactory();
        Session session =sessionFactory.openSession();

        try {
            /* int compteur = ( (Integer) session.iterate(
            "select count(*) from Personnes").next() ).intValue();
            System.out.println("compteur = " + compteur);*/
            String SQL_QUERY = "select count(*) from Personnes
p group by p.ville";
            Query query =session.createQuery(SQL_QUERY);
            for (Iterator it = query.iterate(); it.hasNext();) {
                it.next();
                count++;
            }
            System.out.println("Total rows: " + count);

```

```
SQL_QUERY = "select min(p.Salaire) from Personnes p";
```

```
query = session.createQuery(SQL_QUERY);
List list = query.list();
System.out.println("Min Salaire: " + list.get(0));
```

```
String SQL_QUERY = "select p.ville, sum(p.Salaire)
from Personnes p group by p.ville";
```

```
Query query = session.createQuery(SQL_QUERY);
for (Iterator it = query.iterate(); it.hasNext();) {
    Object[] row = (Object[]) it.next();
    System.out.println("IdPersonne: " + row[0]);
    System.out.println("Somme Salaire: " + row[1]);
}

} finally {
    session.close();
}

sessionFactory.close();
}
```

On peut aussi parcourir les occurrences comme suit :

```
Query query = session.createQuery("from Student");
```

```
List<student> students = query.list();
for(Student student : students)
{
    System.out.println("Nom: "+student.getNom()+", Adresse: "+student.getAdresse()+", Age: "+student.getAge());
}
```

Requêtes qui retournent un ensemble de propriétés (projection)

```
String SQL_QUERY = "Select p.Id,p.nom from Personne s p where p is not null";
Query query = session.createQuery(SQL_QUERY);
for(Iterator it=query.iterate();it.hasNext();){
// utiliser un tableau d'objets
Object[] row = (Object[]) it.next();
System.out.println("ID: " + row[0]);
System.out.println("Name: " + row[1]);
System.out.println("Amount: " + row[2]);
}
```

Expressions

Les expressions permises dans la clause **where** incluent la plupart des choses que vous pouvez utiliser en SQL :

- opérateurs mathématiques +, -, *, /
- opérateur de comparaison binaire =, >=, <=, <>, !=, like
- opérateurs logiques and, or, not
- in, not in, between, is null, is not null, is empty, is not empty,
- concatenation de chaîne de caractères ...||... ou concat(...,...)
- current_date(), current_time(), current_timestamp()

- second(...), minute(...), hour(...), day(...), month(...), year(...),
- §N'importe quel fonction ou opérateur substring(), trim(), lower(), upper(), length(), locate(), abs(), sqrt(), mod() str() pour convertir des valeurs numériques ou temporelles vers une chaîne de caractères
- Les opérations some, all, exists, any, in.
- fonction scalaire comme sign(), trunc(), rtrim(), sin() etc..

Req1: from Employee e where e.adresse="Tana" and e.salaire >1000 order by e.nom;

Select upper(e.nom) from Employee e where e.adresse="Tana" and e.salaire >1000;

Req2: from Employee e where [not] e.salaire between 1000 and 2000;

Req3: from Employee e where [not] e.nom in ('Rabe', 'Rivo', 'Jean')

Exemple de sous-sélection

Req4: String sql_query = "select p.name,p.salaire from Employe where p.salaire = (select max(p.salaire) from p)";

Req5: String sql_query = "select e.name,e.salaire from Employe e where e.salaire > ALL | ANY|SOME (select e.salaire from e where adresse='Tana')";
Query query = sess.createQuery(sql_query);
for(Iterator it=query.iterate();it.hasNext();){
Object[] row = (Object[]) it.next();
System.out.print(row[0]); System.out.println("\t\t"+row[1]);
}

Lier des paramètres

- Des méthodes de Query sont fournies pour lier des valeurs à des paramètres nommés ou à des paramètres de style JDBC ?. *Contrairement à JDBC, les numéros des paramètres d'Hibernate commencent à zéro.* Les paramètres nommés sont des identifiants de la forme :**nom** dans la chaîne de caractères de la requête. Les avantages des paramètres nommés sont : les paramètres nommés sont insensibles à l'ordre de leur place dans la chaîne de la requête
- ils peuvent apparaître plusieurs fois dans la même requête

Exemple

//paramètre nommé (préféré)

```
Query q = sess.createQuery("from Employee e where e.nom=:name and e.solde<:solde");
q.setString("name", "Fritz");q.setInteger("solde", 100);
Iterator iter = q.iterate();
```

//paramètre positionnel

```
Query q = sess.createQuery("from Employee e where e.nom=? and e.datenais<?");
q.setString(0, "rabe");
```

```
q.setDate(1, new Date());
Iterator iter = q.iterate();
```

//liste de paramètres nommés

```
List names = new ArrayList();
names.add("rivo");
names.add("jean");
Query q = sess.createQuery("from Employee e where
e.nom in (:namesList)");
q.setParameterList("namesList", names);
List liste = q.list();
```

Pagination

Si on a besoin de spécifier le nombre maximum de lignes et/ou la première ligne que on veut récupérer , on peut utiliser des méthodes **setMaxResults** et **setFirstResult** de l'interface Query :

Exemple:

```
Query q = sess.createQuery("from Employee e");
q.setFirstResult(20);
q.setMaxResults(10);
List liste = q.list();
```

Externaliser des requêtes nommées

Vous pouvez aussi définir des requêtes nommées dans le document de mapping. On utilise une section CDATA si votre requête contient des caractères qui pourraient être interprétés comme des éléments XML.

```
<query name="exa.req_liste_emp">
<![CDATA[ from exa.Employee as e where e.nom = ?
and e.age > ? ]>
</query>
```

La liaison de paramètres et l'exécution sont fait par programmation :

```
Query q =
sess.getNamedQuery("exa.req_liste_emp");
q.setString(0, "rivo");q.setInt(1, 20);
List liste= q.list();
```

Les fonctions d'agrégation

```
count( [ distinct | all ] object | object.property )
count(*) (equivalent au count(all ...),
sum ( [ distinct | all ] object.property)
avg( [ distinct | all ] object.property)
max( [ distinct | all ] object.property)
```

Req1:

```
String SQL_QUERY = "select avg(e.salaire) from
Employee e";
Query query = sess.createQuery(SQL_QUERY);
List list = query.list();
System.out.println("Salaire moyen:" + list.get(0));
```

Req 2:

```
Query query=session.createQuery("select
avg(e.salaire), sum(e.salaire), max(e.salaire), count(e)
from Employee e ");
```

```
for (Iterator it = query.iterate(); it.hasNext();)
{
Object[] row = (Object[]) it.next();
System.out.print("AVG: " + row[0]);
System.out.print("SUM: " + row[1]);
System.out.print("MAX: " + row[2]);
System.out.println("COUNT: " + row[3]);
}
```

Req 3: HQL Group By Clause

```
String SQL_QUERY = "select e.ville,count(*),
sum(e.salaire) from Employee e group by
e.ville"; Query query = session.createQuery(SQL_QUE
RY);
for (Iterator it = query.iterate(); it.hasNext();) {
Object[] row = (Object[]) it.next();
System.out.println("Ville: " + row[0]);
System.out.println("Nombre: " + row[1]);
System.out.println("Somme: " + row[2]);
}
```

Associations et jointures

On peut assigner des alias à des entités associées, ou même aux éléments d'une collection de valeurs, en utilisant un **join (jointure)**.

Les types de jointures supportées sont celles de ANSI SQL

- **inner join** (jointure fermée)
- **left outer join** (jointure ouverte par la gauche)
- **right outer join** (jointure ouverte par la droite)

Relation oneToMany bidirectionnelle entre Department , Employee et Phone

```
package onetomany;
import javax.persistence.*;
@Entity
public class Department {
@Id
@GeneratedValue(strategy=GenerationType.AUTO)
private int id;
private String name;
@OneToMany(mappedBy="department")
private Collection<Employee> employees = new
ArrayList<Employee>();
public void addEmployee(Employee employee) {
if (!employees.contains(employee)) {
employees.add(employee);}
}
// setters et getters
}
```

```
package onetomany;
import javax.persistence.*;
@Entity
```



```

public class Employee {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;
    private String name;
    @ManyToOne
    @JoinColumn(name = "DEP_ID")
    private Department department;
    // setters et getters
    @OneToMany(mappedBy="myemployees")
    private Collection<Phone> phones = new
    ArrayList<Phone>();
    public Collection<Phone> getPhones() {return phones;}
    public void setPhones(Collection<Phone> phones) {
    this.phones = phones;}

    public void addPhone(Phone phone) {
    if (!phones.contains(phone)) {
    phones.add(phone); }
    }}

package onetomany;
import javax.persistence.*;
@Entity
public class Phone {
    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private long id;
    private String number;
    private String type;
    @ManyToOne
    @JoinColumn(name = "EMP_ID")
    Employee myemployees;
    public Employee getMyemployees() {
    return myemployees;
    }
    public void setMyemployees(Employee myemployees)
    {
    this.myemployees = myemployees;
    }

    // setters et getters
}

```

hibernate.cfg.xml

```

<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.connection.driver_class">
com.mysql.jdbc.Driver</property>
<property name="hibernate.connection.url">
jdbc:mysql://localhost/base</property>
<property
name="hibernate.connection.username">root</property>
>
<property
name="hibernate.connection.password"></property>
<property
name="dialect">org.hibernate.dialect.MySQLDialect</pr
operty>
<!-- Mapping files -->
    <mapping class="onetomany.Employee"/>

```

```

    <mapping class="onetomany.Department"/>
    <mapping class="onetomany.Phone"/>
</session-factory>
</hibernate-configuration>

```

//INSERTION DES DONNEES

```

Session session = sessionFactory.openSession();
Department dep1 = new Department();
dept1.setName("Q/A");

Employee emp1 = new Employee();
emp1.setName("Jack1");
emp1.setDepartment(dep1);

Employee emp2 = new Employee();
emp2.setName("Jack2");
emp2.setDepartment(dep1);

/* Collection<Employee> emp = new
ArrayList<Employee>();
emp.add(emp1);
emp.add(emp2);
dep1.setEmployees(emp); */
// dep1.addEmployee(emp1);
// dep1.addEmployee(emp2);
Department dep2 = new Department();
dept2.setName("HR");

Employee emp3 = new Employee();
emp3.setName("Mary");emp3.setDepartment(dep2)
;
// dep2.addEmployee(emp3);

Phone phone1 = new Phone();
phone1.setNumber("111");
phone1.setType("T1");
phone1.setMyemployees(emp1);
emp1.addPhone(phone1);

Phone phone2 = new Phone();
phone2.setNumber("222");
phone2.setType("T2");
phone2.setMyemployees(emp2);
emp2.addPhone(phone2);
Phone phone3 = new Phone();
phone3.setNumber("333");
phone3.setType("T3");
phone3.setMyemployees(emp3);
emp3.addPhone(phone3);

session.save(dep1);
session.save(dep2);
session.save(emp1);
session.save(emp2);
session.save(emp3);
    session.save(phone1);
    session.save(phone2);
    session.save(phone3);

```

```
session.getTransaction().commit();
```

REQUÊTAGE

Exemple1:Requêtes qui retournent des tuples

Les requêtes d'Hibernate retournent parfois des tuples d'objets, auquel cas chaque tuple est retourné comme un tableau :

```
Query q = session.createQuery(" [select dept , emp]
from Department d inner join d.employees as e where
e.name like '%ra%' ");
Iterator ite = q.list().iterator();
while ( ite.hasNext() ) {
Object[] tuple = (Object[]) ite.next();
Department dep = (Department) tuple[0];
Employee emp = (Employee) tuple[1];
System.out.print("DEP:" + dep.getName());
System.out.println(" EMP: " + emp.getName());}
```

Exemple 2:

```
List liste= session.createQuery("select p.employees
from Department p ").list();
```

```
for (Iterator it = list.iterator(); it.hasNext();){
Employee p = (Employee) it.next();
System.out.println(p.getName());
}
```

Exemple3:

```
List liste = session.createQuery("select p from
Department p where p.employees.id=1").list();
for (Iterator it = liste.iterator(); it.hasNext();){
Department p = (Department) it.next();
System.out.println(p.getName());
}
```

Exemple 4: Query query=session.createQuery("select d.name, e.name from Department as d **join|inner join| left outer join| right outer join d.employees** as e ");
for (Iterator it = query.iterate(); it.hasNext();){
Object[] row = (Object[]) it.next();
System.out.print("DEPT: " + row[0]);
System.out.println("EMP: " + row[1]);
}

Exemple 5:

```
Query query=session.createQuery("select d.name,
e.name from MyEmployee as e join e.department as
d");
```

```
for (Iterator it = query.iterate(); it.hasNext();){
{
Object[] row = (Object[]) it.next();
System.out.print("DEPT: " + row[0]);
System.out.println("EMPP: " + row[1]);
}
```

Exemple6:

```
List <Employee> liste=session.createQuery("select e
from Department as d join d.employees as e where
d.id=1 ").list();
for (Iterator it = liste.iterator(); it.hasNext();){
Employee p = (Employee) it.next();
System.out.println("NOM EMP:
"+p.getName());
}
```

Exemple 7:

```
List liste=session.createQuery("select COUNT(e) from
Department as d join d.employees as e WHERE
d.id=1").list();
System.out.println("NOMBRE:" + liste.get(0));
```

Exemple 8: Group By

```
Query query=session.createQuery("select d.name,
COUNT(e) from Department as d join d.employees as
e GROUP BY d.id HAVING COUNT(e)>=2");
for (Iterator it = query.iterate(); it.hasNext();){
Object[] row = (Object[]) it.next();
System.out.print("DEPT: " + row[0]);
System.out.println("NOMBRE: " + row[1]);
}
```

Exemple 9: Afficher le nom du département N°5

```
Query q = session.createQuery("select p from
Department p left join p.employees where p.id = :pid");
q.setParameter("pid", 5);
Department d = (Department) q.uniqueResult();
```

```
System.out.print("Nom département: " +
d.getName());
```

Exemple 9: Sous-requête Hibernate (sous-selection)

```
▪ § String sql_query = "select d.name,e.name,e.s
alaire
from Department d join d.employees e where
e.salaire in
(select e.salaire from e where e.salaire > 1500)"
;
String sql_query = "select d.name,e.name,e.sal
aire
from Department d join d.employees e where e.
salaire > ALL | ANY|SOME
(select e.salaire from e where e.salaire > 1500)"
;
Query query = sess.createQuery(sql_query);
for(Iterator it=query.iterate();it.hasNext();){
Object[] row = (Object[]) it.next();
System.out.print(row[0]); System.out.print("\t\t
"+row[1]);
System.out.print("\t"+row[2]);
System.out.println(); }
```

Jointure de 3 entités persistantes

Exemple 1: Liste des employés du département n°5

```
Query query =session.createQuery("select e from
Department d join d.employees e join e.phones p
where d.id=5");
for (Iterator it = query.iterate(); it.hasNext();){
Employee p = (Employee) it.next();
System.out.println(" EMP: " + p.getName());}
```

Exemple 2: Liste des employés avec ses phones du département n°5

```
Query query =session.createQuery("select d.name,
e.name, p.number from Department d join d.employees
e join e.phones p where d.id=5");
for (Iterator it = query.iterate(); it.hasNext();) {
    Object[] row = (Object[]) it.next();
    System.out.print("DEPT: " + row[0]);
    System.out.print(" EMP: " + row[1]);
    System.out.println(" PHONE: " + row[2]);}
```

Exemple 3: En HQL, il est possible de charger les données d'objets dépendants en utilisant la clause "**left join fetch**". Une jointure "fetchée" (rapportée) permet d'initialiser les associations ou collections de valeurs en même temps que leur objet parent, le tout n'utilisant qu'un seul Select. Une jointure "fetchée" n'a pas besoin des alias puisque les objets associés ne sont pas retournés directement dans le résultat de la requête mais l'on peut y accéder via l'objet parent

- § Query query =session.createQuery(" from Employee e **join fetch** e.department where e.department.id=5"); § Query query =session.createQuery("select e from Employee e **inner join fetch** e.department where e.department.id=5"); Query query =session.createQuery("select e from Employee e **inner join fetch** e.department join fetch e.phones where e.department.id=5");

```
List <Employee> liste= query.list();
for (Iterator it = liste.iterator(); it.hasNext();){
    Employee p = (Employee) it.next();
    System.out.println(" EMP: "+p.getName());}
```

Exemple 3:

```
List list =session.createQuery("select d.name,
e.name, p.number from Department d, Employee e,
Phone p WHERE d.id=e.DEP_ID AND
e.id=p.EMP_ID").list();
```

```
for (Iterator it = list.iterator(); it.hasNext();)
{
    Object[] row = (Object[]) it.next();
    System.out.print("DEPT: " + row[0]);
    System.out.print("EMP: " + row[1]);
    System.out.println("PHONE: " + row[2]);
}
```

Hibernate Native SQL

Hibernate permet d'exécuter une requête native écrite sous forme SQL (Native SQL) comme les opérations create, update, delete et select

Exemple 1:

```
List list =session.createQuery("select * from
Department").list();
List list =session.createQuery("SELECT ID, NAME
FROM Department").list();
```

On peut indiquer les colonnes avec les types qu'on veut retourner en utilisant **addscalar**. List list = session.createQuery("SELECT * FROM Employee").**addScalar**("ID",

```
Hibernate.INTEGER).addScalar("NAME",
Hibernate.STRING).addScalar("DATENAIS",
Hibernate.DATE).addScalar("ADRESSE").list();
for (Iterator it = list.iterator(); it.hasNext();)
{ Object[] row = (Object[]) it.next();
    System.out.print(" ID: " + row[0]);
    System.out.println(" NOM: " + row[1]);.....
}
```

Exemple 2:

```
Query query = session.createQuery("DELETE
FROM Employee WHERE id= 24"); Query query =
session.createQuery("UPDATE Employee SET
name='Jean' WHERE id= 1"); Query queryy =
session.createQuery("INSERT INTO Employee
(nom,adresse) VALUES('koto','Tana') ");
int records = query.executeUpdate();
System.out.println(records +" record (s) has been
updated");
```

Exemple 3:

```
String sql ="select sum(p.salaire)
as tot, avg(p.salaire) as mean from Employee p";
Query query = session.createQuery(sql)
.addScalar("tot", Hibernate.DOUBLE).addScalar("mean",
Hibernate.DOUBLE);
//Double [] amount = (Double []) query.uniqueResult();
Object [] amount = (Object [])query.uniqueResult();
System.out.println("sal moy: " + amount[0]);
System.out.println("sal tot: " + amount[1]);
```

Exemple 4:

```
Double max = (Double) session.createQuery("select
max(e.salaire) as maxsal from Employee e")
.addScalar("maxsal", Hibernate.DOUBLE)
.uniqueResult();
```

```
System.out.println("Average of Invested Amount: " +
max);
```

Exemple 5:

```
List listes=session.createQuery("select d.name,
e.name from Department d JOIN Employee e ON
d.id=e.DEP_ID WHERE d.id=1").list();
```

```
List listes=session.createQuery("select d.name,
e.name from Department d, Employee e WHERE
d.id=e.DEP_ID").list();
```

```
for (Iterator it = listes.iterator(); it.hasNext();)
{
    Object[] row = (Object[]) it.next();
    System.out.print("DEPT: " + row[0]);
    System.out.println("EMP: " + row[1]);
}
```

Exemple 6:

```
List liste2=session.createQuery("select d.name,
COUNT(*) AS nb from Department d JOIN Employee e
ON d.id=e.DEP_ID GROUP BY d.id HAVING nb
>=2").list();
```

```
for (Iterator it = liste2.iterator(); it.hasNext();)
{
    Object[] row = (Object[]) it.next();
    System.out.print("DEPT: " + row[0]);
    System.out.println("NOMBRE: " + row[1]);
}
```

Exemple 7:

```
List liste=session.createQuery("select COUNT(*)
from Department d JOIN Employee e ON
d.id=e.DEP_ID WHERE d.id=1").list();
System.out.println("NOMBRE:"+liste.get(0));
```

Requête d'entité

Une requête native retournant un objet d'entité via **addEntity()**.

Exemple 1:

```
List list =session.createQuery("SELECT * FROM
Department").addEntity(Department.class).list();
```

```
List list =session.createQuery("SELECT id,name
FROM Department").addEntity(Department.class).list();
```

```
for (Iterator it = list.iterator(); it.hasNext();)
{
    Department p = (Department) it.next();
    System.out.println(p.getName());
}
```

Exemple2:

```
/* Selecting all the objects from person table */
```

```
List
liste = session.createQuery("select {p.*} from
Personne p") .addEntity("p", Personne.class).list();
for (Iterator it = liste.iterator(); it.hasNext();)
{
    Personne pers = (Personne) it.next();
    System.out.println("Name: " + pers.getNom());
}
```

La requête native SQL peut avoir des **paramètres**:

Exemple 1:

```
Query query = sess.createQuery("SELECT * FROM
Employee WHERE NAME like ?")
.addEntity(Employee.class);
List liste = query.setString(0, "ra%").list();
```

Exemple 2:

```
Query query = sess.createQuery("SELECT * FROM
Employee WHERE NAME like
:name").addEntity(Employee.class);
List liste = query.setString("name", "ra%").list();
```

Hibernate Named Query

Une requête nommée est déclarée dans un fichier mapping xml file(.hbm files) et accessible dans une application.
Cependant, leurs définitions sont globales au scope de la session factory/entity manager factory. Une requête nommée est définie par son nom et la chaîne de caractères de la requête réelle.

```
<hibernate-mapping>
<class name="Contact" table="contact">
<id name="id" type="long" column="ID">
<generator class="increment"/>
</id>
<property name="nom">
<column name="nom" />
</property>
</class>
<query name="findContactByFirstName">
<![CDATA[from Contact c where c.nom = :nom]]>
</query>
</hibernate-mapping>
```

Appel de la requête nommée dans le code

Avec la méthode **getNamedQuery** qui prend le nom de la requête comme paramètre.

Query query =

```
session.getNamedQuery("findContactByFirstName")
.setString("nom", "Ravi");
List results = query.list();
System.out.println("No of Results: " + results.size());
```

Vous pouvez mapper des requêtes JPA-QL/HQL en utilisant les annotations **@NamedQuery** et **@NamedQueries** définies au niveau de la classe ou dans un fichier JPA XML.

@Entity

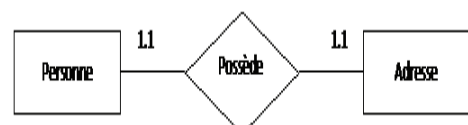
```
@NamedQuery(name="liste_emp", query="select e from
Employee e where e.salaire >= :salaire")
```

```
public class Employee { ...}
```

```
Query q = session.getNamedQuery("liste_emp");
q.setString( "salaire", 1000 );
List results = q.list();
System.out.println("No of Results: " + results.size()); ...
```

3. Les relations un / un

Dans ce type de relation, deux entités sont liées de façon à n'avoir qu'une seule et unique occurrence l'une pour l'autre.



Chaque personne ne peut avoir qu'une seule adresse et une adresse ne peut appartenir qu'à une seule personne.

Cette relation peut se traduire de plusieurs manières dans la base de données :

- Une seule table qui contient les données de la personne et son adresse
- Deux tables, une pour les personnes et une pour les adresses avec une clé primaire partagée
- Deux tables, une pour les personnes et une pour les adresses avec une clé étrangère

Il y a plusieurs façons de traiter ce cas avec une ou deux tables dans la base de données et Hibernate :

- Deux tables et une relation One-to-One d'Hibernate
- Une seule table avec un Component d'Hibernate

3.1. Le mapping avec un Component

Comme une personne ne peut avoir qu'une seule adresse, il est préférable pour des raisons de performance de stocker les données des deux entités dans une seule et même table. Ceci évite d'avoir à faire une jointure lors de l'accès aux données des deux entités.

La description de la table personne est la suivante :

Résultat :
<pre>CREATE TABLE `personne` (`id` int(11) NOT NULL auto_increment, `Nom` varchar(255) NOT NULL default "", `cp_adr` varchar(5) default NULL, `ville_adr` varchar(80) default NULL, PRIMARY KEY (`id`)) ENGINE=InnoDB ;</pre>

La configuration dans le fichier de mapping

Les classes qui encapsulent l'entité personne et les données de l'adresse sont de simples POJO.

Exemple :

```
package com.test.hibernate;

public class Personne {

    private Long id;
    private String nom;

    private Adresse adresse;
```

```
public Personne(String nom, Adresse adresse) {
    this.nom = nom;
    this.adresse = adresse;
}

public Personne() {
}

public String getNom() {
    return nom;
}

public void setNom(String nom) {
    this.nom = nom;
}

public String getPrenom() {
    return prenom;
}

public void setPrenom(String prenom) {
    this.prenom = prenom;
}

public Long getId() {
    return id;
}

// Attention le setter est requis par Hibernate
public void setId(Long id) {
    this.id = id;
}

public Adresse getAdresse() {
    return adresse;
}

public void setAdresse(Adresse adresse) {
    this.adresse = adresse;
}

@Override
public String toString() {
    return this.id + " : " + this.nom + " " + this.prenom;
}
```

La classe Adresse ne possède pas de champ de type identifiant.

Exemple :

```
package com.test.hibernate;

public class Adresse {

    private Long id;
    private String cp;
    private String ville;
    public Adresse(String cp, String ville) {
        super();
        this.cp = cp;
        this.ville = ville;
    }
}
```

```
public Adresse() {
}
//
// getter et setter sur les champs de la classe
//
}
```

Les données de l'adresse sont encapsulées dans une classe Adresse : la définition des champs de cette classe est faite dans un élément de type component du fichier de mapping de l'entité Personne (Personne.hbm.xml).

Exemple :

```
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping
DTD//EN"
"http://hibernate.sourceforge.net/
hibernate-mapping-3.0.dtd">
<hibernate-mapping>
  <class name="com.jmdoudoux.test.
hibernate.Personne"
table="Personne">
    <id name="id"
column="id">
        <generator class="increment" />
    </id>
    <property name="nom" column="Nom" />
    <component name="adresse" class="
com.test.hibernate.Adresse">
        <property name="cp"
column="cp_adr" />
        <property name="ville"
column="ville_adr" />
    </component>
    </class>
</hibernate-mapping>
```

Le fichier de configuration d'Hibernate définit les paramètres de connexion à la base de données et le fichier de mapping de l'entité Personne.

Exemple :

```
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property
name="connection.url">jdbc:mysql://localhost/
mabdd</property>
    <property
name="connection.username">root</property>
    <property name="connection.password"></property>
    <property name="connection.driver_class">
```

```
com.mysql.jdbc.Driver</property>
    <property name="dialect">
org.hibernate.dialect.MySQLDialect</property>
    <property name="transaction.factory_class">
org.hibernate.transaction.JDBCTransactionFactory</prop
erty>
    <property name="current_session_context_class">
thread</property>
    <property name="hibernate.show_sql">true</property>
    <mapping resource=
"com/jmdoudoux/test/hibernate/Personne.hbm.xml"></ma
pping>
  </session-factory>
</hibernate-configuration>
```

L'application de test est basique :

- créer une instance de type Adresse,
- créer une instance de type personne,
- et sauvegarder la personne en base de données

Exemple :

```
package com.test.hibernate;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class TestHibernate15 {

    public static void main(String args[]) {
        SessionFactory sessionFactory = new
Configuration().configure().buildSessionFactory();

        Transaction transaction = null;
        Session session = sessionFactory.openSession();
        try {
            transaction = session.beginTransaction();

            Adresse adresse = new Adresse("cp_1", "ville_1");
            Personne personne = new Personne("nom_1", ,
adresse);

            session.save(personne);
            transaction.commit();

            System.out.println("La nouvelle personne a ete
enregistree");

        } catch (Exception e) {
            transaction.rollback();
            e.printStackTrace();
        } finally {
            session.close();
        }

        sessionFactory.close();
    }
}
```

Lorsque la personne est enregistrée dans la base de données, son adresse l'est aussi dans la table Personne.

Résultat :
mysql> select * from personne; Id Nom cp_adr ville_adr 1 nom_1 cp_1 ville_1

3.2. Le mapping avec une relation One-to-One avec clé primaire partagée

La relation repose sur deux tables distinctes : une pour les personnes et une pour les adresses.

Chacune des deux tables possède un identifiant qui est sa clé primaire. La particularité est que la valeur des clés primaires est partagée entre les deux tables. L'identifiant de la table adresse n'est pas auto incrémenté et correspond à la valeur de l'identifiant de la table personne.

Hibernate ne sait pas gérer seul ce type de mapping : il sera nécessaire de l'aider en utilisant un mapping bidirectionnel qui permettra à Hibernate de connaître la valeur de l'identifiant de la personne à utiliser comme valeur de l'identifiant pour l'adresse afin que celui-ci corresponde.

La description de la table personne est la suivante :

Résultat :
CREATE TABLE `personne` (`id` bigint(20) NOT NULL auto_increment, `Nom` varchar(255) NOT NULL default "", PRIMARY KEY (`id`)) ENGINE=InnoDB;

La description de la table adresse est la suivante :

Résultat :
CREATE TABLE `adresse` (`id` bigint(20) NOT NULL default '0', `cp_adr` varchar(5) default NULL, `ville_adr` varchar(80) default NULL, PRIMARY KEY (`id`)) ENGINE=InnoDB;

La configuration dans le fichier de mapping

Les classes qui encapsulent les entités personne et adresse sont de simples POJO.

Exemple :
package com.test.hibernate; public class Adresse {

<pre>private Long id; private String cp; private String ville; private Personne personne; public Adresse(String cp, String ville) { super(); this.cp = cp; this.ville = ville; } public Adresse() { } public Long getId() { return id; } // setter requis par Hibernate public void setId(Long id) { this.id = id; } public Personne getPersonne() { return personne; } public void setPersonne(Personne personne) { this.personne = personne; } // // getter et setter sur les autres champs de la classe // }</pre>
--

Exemple :

<pre>package com.test.hibernate; public class Personne { private Long id; private String nom; private String prenom; private String dateNais; private Adresse adresse; public Personne(String nom, Adresse adresse) { this.nom = nom; this.adresse = adresse; } public Personne() { } public Long getId() { return id; } // Attention le setter est requis par Hibernate public void setId(Long id) { this.id = id; } }</pre>

```

}

public Adresse getAdresse() {
    return adresse;
}

public void setAdresse(Adresse adresse) {
    this.adresse = adresse;
}

//
// getter et setter sur les autres champs de la classe
//

@Override
public String toString() {
    return this.id + " : " + this.nom + " " + this.prenom;
}
}
    
```

Le fichier de mapping de l'entité Personne (Personne.hbm.xml) contient un élément fils <one-to-one> pour définir la relation entre Personne et Adresse.

Exemple :

```

<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
  <class name="
com.test.hibernate.Personne" table="Personne">
    <id name="id" column="id">
      <generator class="increment" />
    </id>
    <property name="nom"
column="Nom" />
    <one-to-one name="adresse"
class="com.test.hibernate.Adresse"
      cascade="save-update" />
    </class>
  </hibernate-mapping>
    
```

Le fichier de mapping de l'entité Adresse (Adresse.hbm.xml) possède plusieurs caractéristiques liées au type de la relation utilisée avec l'entité Personne :

- Le champ identifiant id est défini avec un générateur de type foreign avec un paramètre qui précise que la valeur sera celle de l'identifiant du champ personne
- La relation inverse avec Personne est définie avec un tag <one-to-one> avec l'attribut constrained ayant la valeur true

Exemple :

```

<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping
    
```

```

PUBLIC
"-//Hibernate/Hibernate Mapping
DTD//EN"
"http://hibernate.sourceforge.net/
hibernate-mapping-3.0.dtd">
<hibernate-mapping>
  <class name="
com.jmdoudoux.test.hibernate.Adresse"
table="Adresse">

    <id name="id" column="Id">
      <generator class="foreign">
        <param name="property">personne</param>
      </generator>
    </id>
    <property name="cp" column="cp_adr" />
    <property name="ville" column="ville_adr" />
    <one-to-one name="personne" class="
com..test.hibernate.Personne"
      constrained="true" />
    </class>
  </hibernate-mapping>
    
```

Le fichier de configuration d'Hibernate définit les paramètres de connexion à la base de données et les deux fichiers de mapping des entités.

Exemple :

```

<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration
PUBLIC
"-//Hibernate/Hibernate Configuration
DTD 3.0//EN"
"http://hibernate.sourceforge.net/
hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="connection.url">
jdbc:mysql://localhost/mabdd</property>
    <property name="connection.username">
root</property>
    <property name="connection.password"></property>
    <property name="connection.driver_class">
com.mysql.jdbc.Driver</property>
    <property name="dialect">
org.hibernate.dialect.MySQLDialect </property>
    <property name="transaction.factory_class">
org.hibernate.transaction.JDBCTransactionFactory
</property>
    <property name="current_session_context_class">
thread</property>
    <property name="hibernate.show_sql">true</property>
    <mapping resource="com/test/
hibernate/Personne.hbm.xml">
</mapping>
    <mapping resource="com/test/
hibernate/Adresse.hbm.xml">
</mapping>

  </session-factory>
</hibernate-configuration>
    
```


L'application de test est basique :

- créer une instance de type Adresse,
- créer une instance de type Personne,
- assurer le bon fonctionnement du lien bidirectionnel en fournissant une référence de l'objet Personne à l'instance de l'adresse. Ceci doit être fait manuellement car Hibernate ne prend pas en charge automatiquement les liens bidirectionnels
- et sauvegarder la personne en base de données

Exemple :

```
package com.test.hibernate;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class TestHibernate11 {

    public static void main(String args[]) {
        SessionFactory sessionFactory = new
Configuration().configure().buildSessionFactory();
        Transaction transaction = null;

        Session session = sessionFactory.openSession();

        try {
            transaction = session.beginTransaction();

            Adresse adresse = new Adresse("cp_1", "ville_1");
            Personne personne=new Personne("nom_1",
adresse);

            adresse.setPersonne(personne);

            session.save(personne);
            transaction.commit();

            System.out.println("La nouvelle personne a ete
enregistree");

        } catch (Exception e) {
            transaction.rollback();
            e.printStackTrace();
        } finally {
            session.close();
        }

        sessionFactory.close();
    }
}
```

Lorsque la personne est enregistrée dans la base de données, son adresse l'est aussi avec comme identifiant la valeur de l'identifiant de la personne.

Résultat :

```
mysql> select * from personne;
```

```
mysql> select * from personne;
| Id | Nom
| 1 | nom_1
1 row in set (0.00 sec)
```

```
mysql> select * from adresse;
| Id | cp_adr | ville_adr
| 1 | cp_1 | ville_1
```

3.3. Le mapping avec une relation many-to-One avec clé étrangère

La relation repose sur deux tables distinctes : une pour les personnes et une pour les adresses

Chacune des deux tables possède son propre identifiant et la relation entre les deux tables est assurée par une clé étrangère de la table personne vers la table adresse.

Hibernate sait gérer seul ce type de mapping si il est unidirectionnel.

La description de la table personne est la suivante :

Résultat :

```
CREATE TABLE `personne` (
  `Id` int(11) NOT NULL auto_increment,
  `Nom` varchar(255) NOT NULL default "",
  `adresse_id` int(11) NOT NULL default '0',
  PRIMARY KEY (`Id`)) ENGINE=InnoDB ;
```

La description de la table adresse est la suivante

Résultat :

```
CREATE TABLE `adresse` (
  `id` bigint(20) NOT NULL auto_increment,
  `cp_adr` varchar(5) default NULL,
  `ville_adr` varchar(80) default NULL,
  PRIMARY KEY (`id`)) ENGINE=InnoDB ;
```

. La configuration dans le fichier de mapping

Les classes qui encapsulent les entités personne et adresse sont de simples POJO.

Exemple :

```
package com.test.hibernate;

public class Personne {

    private Long id;
    private String nom;

    private Adresse adresse;

    public Personne(String nom, Adresse adresse) {
```

```

this.nom = nom;

this.adresse = adresse;
}

public Personne() {
}

public Long getId() {
    return id;
}

// Attention le setter est requis par Hibernate
public void setId(Long id) {
    this.id = id;
}

public Adresse getAdresse() {
    return adresse;
}

public void setAdresse(Adresse adresse) {
    this.adresse = adresse;
}

//
// getter et setter sur les autres champs de la classe
//

@Override
public String toString() {
    return this.id + " : " + this.nom + " " + this.prenom;
}
}

Exemple :

package com..test.hibernate;
public class Adresse {
    private Long id;
    private String cp;
    private String ville;
    public Adresse(String cp, String ville) {
        super();
        this.cp = cp;
        this.ville = ville;
    }
    public Adresse() {
    }
    public Long getId() {
        return id;
    }
    // setter requis par Hibernate
    public void setId(Long id) {
        this.id = id;
    }
}

//
// getter et setter sur les autres champs de la classe
//

```

Le fichier de mapping de l'entité Personne (Personne.hbm.xml) contient un élément fils <many-to-one> pour définir la relation entre Personne et Adresse : l'unicité de la relation est cependant garantie par la valeur true de l'attribut unique. Il faut aussi utiliser une propriété column pour préciser la colonne qui va contenir la clé étrangère vers la table adresse.

Exemple :

```

<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="com.jmdoudoux.test.hibernate.Personne"
table="Personne">
        <id name="id" column="Id">
            <generator class="increment" />
        </id>
        <property name="nom" column="Nom" />
        <property name="prenom" column="Prenom" />
        <property name="dateNais" column="DateNais" />
        <many-to-one name="adresse"
class="com..test.hibernate.Adresse"
column="adresse_id" cascade="all" unique="true" />
    </class>
</hibernate-mapping>

```

Le fichier de mapping de l'entité Adresse (Adresse.hbm.xml) ne contient aucune particularité.

Exemple :

```

<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC
"-//Hibernate/Hibernate Mapping DTD//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="com.jmdoudoux.test.
hibernate.Adresse" table="Adresse">
        <id name="id">
            <generator class="increment" />
        </id>
        <property name="cp" column="cp_adr" />
        <property name="ville" column="ville_adr" />
    </class>
</hibernate-mapping>

```

Le fichier de configuration d'Hibernate définit les paramètres de connexion à la base de données et les deux fichiers de mapping des entités.

Exemple :

```

<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-configuration-
3.0.dtd">
<hibernate-configuration>
<session-factory>

```

```

<property name="connection.url">
jdbc:mysql://localhost/mabdd</property>
<property name="connection.username">
root</property>
<property name="connection.password">
</property>
<property name="connection.driver_class">
com.mysql.jdbc.Driver</property>
<property name="dialect">
org.hibernate.dialect.MySQLDialect</property>
<property name="transaction.factory_class">
org.hibernate.transaction.
JDBCTransactionFactory</property>
<property name="current_session_context_class">
thread</property>
<property name="hibernate.show_sql">
true</property>
<mapping resource="com/test/hibernate/
Personne.hbm.xml"></mapping>
<mapping resource="com/test/hibernate/
Adresse.hbm.xml"></mapping>

</session-factory>
</hibernate-configuration>

```

L'application de test est basique :

- créer une instance de type Adresse,
- créer une instance de type Personne,
- et sauvegarder la personne en base de données

Exemple :

```

package com.jmdoudoux.test.hibernate;

import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class TestHibernate12 {

    public static void main(String args[]) {
        SessionFactory sessionFactory = new
Configuration().configure().buildSessionFactory();
        Transaction transaction = null;
        int index = 4;

        Session session = sessionFactory.openSession();

        try {
            transaction = session.beginTransaction();

            Adresse adresse = new Adresse("cp_1", "ville_1");
Personne personne=new Personne("nom_1", adresse);

            session.save(personne);
            transaction.commit();

            System.out.println("La nouvelle personne a ete
enregistree");

        } catch (Exception e) {

```

```

        transaction.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }

    sessionFactory.close();
}
}

```

Lorsque la personne est enregistrée dans la base de données, son adresse l'est aussi. La nouvelle occurrence de la table Personne et de la table Adresse possède chacun leur propre identifiant et celui de l'adresse est reporté dans le champ adresse_id de la table Personne.

Résultat :

```

mysql> select * from personne;
| Id | Nom   | adresse_id |
+---+-----+-----+
| 1 | nom_1 |      8     |
+---+-----+-----+
1 row in set (0.82 sec)

```

```

mysql> select * from adresse;
+---+-----+-----+
| id | cp_adr | ville_adr |
+---+-----+-----+
| 8 | cp_1  | ville_1   |
+---+-----+-----+
1 row in set (0.00 sec)

```

4. L'API Criteria

En plus du langage HQL pour réaliser des requêtes d'extraction de données, Hibernate propose une API qui permet de construire des requêtes pour interroger la base de données. L'API Criteria d'Hibernate propose donc une alternative à HQL sous la forme d'une API.

Le HQL possède une syntaxe dérivée de celle de SQL dans laquelle les notions relationnelles sont remplacées par des notions objets. Ceci oblige les développeurs à utiliser une syntaxe proche de celle du SQL.

L'API Criteria Query propose des objets pour définir les critères d'une requête ce qui permet aux développeurs de les définir d'une manière orienté objet plutôt que d'utiliser le HQL.

L'API Criteria propose donc d'avoir une approche orientée objet pour permettre de définir des requêtes pour obtenir des données. L'utilisation de cette API permet d'avoir un meilleur contrôle grâce à la compilation.

Cette API permet de facilement combiner de nombreux critères optionnels pour créer une requête : elle est particulièrement adaptée pour créer dynamiquement des requêtes à la volée comme c'est le cas par exemple

pour des requêtes effectuant des recherches multicritères à partir d'informations fournies par l'utilisateur.

Elle offre pour la plupart des fonctionnalités une approche bi directionnelle :

- appliquer un critère en désignant la propriété sur lequel il s'applique
- appliquer un critère sur une propriété

Elle propose des classes et des interfaces qui encapsulent les fonctionnalités de SQL dont les principales sont :

- Criteria
- Criterion
- Restrictions
- Projection
- Order

L'interface org.hibernate.criterion.Criteria est le point d'entrée pour utiliser l'API Criteria. Elle permet de définir une requête à partir de critères pour retrouver des données.

Exemple :

```
select * from Personne
```

Voici le code équivalent avec l'API Criteria :

Exemple :

```
List personnes = session.createCriteria(Personne.class).list();
```

Les critères de recherche permettant de restreindre les données retournées par la requête sont définis par l'interface org.hibernate.criterion.Criterion. Le type Criterion encapsule un élément de la clause "where" de la requête SQL qui sera générée. La classe org.hibernate.criterion.**Restrictions** est une fabrique qui propose des méthodes statiques pour créer des instances de type Criterion

Depuis la version 3.x d'Hibernate, il est préférable d'utiliser la classe **Restrictions** à sa classe fille Expressions.

Exemple :

```
import org.hibernate.*;
import org.hibernate.criterion.*;
import org.hibernate.cfg.*;
```

```
Criteria criteria = session.createCriteria(Personne.class);
Criterion critere = Restrictions.eq("id", 2l);
//criteria.add(Restrictions.like("Nom", "%a%")); //Like condition
```

```
// criteria.add(Expression.between("Salaire", new
Integer(1000), new Integer(2500))); // Between condition

criteria.add(critere);
criteria.setMaxResults(5); //Restricts the max rows to 5
List personnes = criteria.list();
System.out.println("nb personnes = " + personnes.size());
Iterator it = personnes.iterator();
while (it.hasNext()) {
    Personne personne = (Personne) it.next();
    System.out.println("Personne : " + personne);
}
List liste = criteria.list();
for(Iterator it = liste.iterator();it.hasNext();){
    Personne pers = (Personne) it.next();
    System.out.println("ID: " + pers.getId());
    System.out.println("Name: " + pers.getNom());
}
}
```

L'interface org.hibernate.criterion.Projection encapsule un champ en réponse de la requête (un champ dans la clause "select" de la requête SQL).

La classe org.hibernate.criterion.Projections est une fabrique pour les instances de type Projection.

Exemple :

```
SELECT nomFROM Personne
```

Exemple :

```
List noms = session.createCriteria(Personne.class)
.setProjection(Projections.property("nom")).list();
System.out.println("nb personnes = " + noms.size());
Iterator it = noms.iterator();
while (it.hasNext()) {
    String nom = (String) it.next();
    System.out.println("Personne : " + nom);
}
```

La classe Order encapsule une clause SQL "order by".

L'API Criteria permet de définir à la volée des recherches de données multi-critères complexes.

Exemple :

```
public List rechercher(Session session,
String nom,
String prenom,
Date dateDeb,
Date dateFin,
Integer taille) {
Criteria criteria = session.createCriteria(Personne.class);
if (dateDeb != null) {
    criteria.add(Restrictions.ge("dateNais", dateDeb));
}
if (dateFin != null) {
    criteria.add(Restrictions.le("dateNais", dateFin));
}
```

```

}
if (nom != null) {
    criteria.add(Restrictions.eq("nom", nom));
}
if (prenom != null) {
    criteria.add(Restrictions.eq("prenom", prenom));
}
if (taille != null) {
    criteria.add(Restrictions.eq("taille", taille));
}
List resultat = criteria.list();

return resultat;
}

```

4.1.. L'interface Criteria

L'interface org.hibernate.Criteria propose des fonctionnalités pour encapsuler une requête composée de critères.

Une instance de l'interface Criteria est obtenue en invoquant la méthode createCriteria() de la session hibernate. Elle attend en paramètre la classe d'une entité sur laquelle les critères vont s'appliquer.

L'interface Criteria propose différentes méthodes pour construire les critères d'interrogation sur une classe persistante.

Méthode	Rôle
Criteria add(Criterion criterion)	ajouter un critère Criteria
addOrder(Order order)	ajouter un ordre de tri
Criteria createAlias(String associationPath, String alias)	ajouter une jointure en lui assignant un alias
Criteria createAlias(String associationPath, String alias, int joinType)	créer un objet de type Criteria pour une entité donnée en précisant son type et en lui assignant un alias
Criteria createCriteria(String associationPath)	créer un objet de type Criteria pour une entité donnée
Criteria createCriteria(String associationPath, int joinType)	créer un objet de type Criteria pour une entité donnée en précisant son type de jointure
Criteria createCriteria(String associationPath, String alias)	créer un objet de type Criteria pour une entité donnée en lui assignant un alias
Criteria createCriteria(String associationPath, String alias, int joinType)	créer un objet de type Criteria pour une entité donnée en précisant son type, en lui assignant un alias et en précisant son type de jointure

String getAlias()	obtenir l'alias de l'entité encapsulée dans le Criteria
List list()	obtenir les résultats de la requête
ScrollableResults scroll()	obtenir les résultats comme une instance de type ScrollableResults
ScrollableResults scroll(ScrollMode scrollMode)	obtenir les résultats comme une instance de type ScrollableResults en précisant le mode de parcours
Criteria setCacheable(boolean cacheable)	activer ou non la mise en cache des résultats de la requête
Criteria setCacheMode(CacheMode cacheMode)	modifier le mode de mise en cache des résultats de la requête
Criteria setCacheRegion(String cacheRegion)	préciser le nom de la région du cache à utiliser pour stocker les résultats de la requête
Criteria setComment(String comment)	ajouter un commentaire à la requête SQL
Criteria setFetchMode(String associationPath, FetchMode mode)	préciser le mode de récupération des données dans le cas d'une association entre deux entités
Criteria setFetchSize(int fetchSize)	préciser le nombre d'occurrences retournées par la requête
Criteria setFirstResult(int firstResult)	préciser la première occurrence qui sera retournée
Criteria setFlushMode(FlushMode flushMode)	préciser le mode de flush de la requête
Criteria setLockMode(LockMode lockMode)	préciser le mode de verrou de l'entité
Criteria setLockMode(String alias, LockMode lockMode)	préciser le mode de verrou pour l'entité dont l'alias est fourni en paramètre
Criteria setMaxResults(int maxResults)	assigner un nombre maximum d'occurrences retournées dans le résultat
Criteria setProjection(Projection projection)	préciser le contenu du résultat de la requête
Criteria setResultTransformer(ResultTransformer resultTransformer)	préciser une stratégie de traitement des résultats de la requête
Criteria setTimeout(int timeout)	assigner un timeout à la requête
Object uniqueResult()	ne renvoyer qu'une seule instance

en résultat de la requête ou null si la requête ne renvoie aucun résultat.

Exemple :

```
Criteria criteria = session.createCriteria(Personne.class);
criteria.setMaxResults(10);
List personnes = criteria.list();
```

4.2. L'interface Criterion

L'interface org.hibernate.criterion.Criterion définit les méthodes pour un élément qui va permettre de définir un critère à appliquer dans la requête. Chaque instance d'un critère doit être ajouté aux critères de la requête en utilisant la méthodes add() de l'instance de type Criteria.

L'API propose plusieurs fabriques qui permettent d'instancier les différents objets qui vont définir le contenu de la requête.

Exemple :

```
List personnes =
session.createCriteria(Personne.class)
.add(Restrictions.like("nom", "Dup%"))
.add(Restrictions.gt("taille", new Integer(180)))
.addOrder(Order.asc("age")).list();
```

La classe org.hibernate.criterion.Restrictions est une fabrique qui propose des méthodes pour obtenir différentes instances de Criterion.

Exemple :

```
Calendar cal = Calendar.getInstance();
cal.set(1980, Calendar.JANUARY, 01);
Date dateDeb = cal.getTime();
cal.set(1980, Calendar.DECEMBER, 31);
Date dateFin = cal.getTime();
personnes = session.createCriteria(Personne.class)
.add(Restrictions.ge("dateNais", dateDeb))
.add(Restrictions.le("dateNais", dateFin))
.addOrder(Order.asc("dateNais"))
.setFirstResult(0)
.setMaxResults(10).list();
```

L'interface Criterion possède de nombreuses interfaces filles : AbstractEmptinessExpression, BetweenExpression, Conjunction, Disjunction, EmptyExpression, Example, ExistsSubqueryExpression, IdentifierEqExpression, IlikeExpression, InExpression, Junction, LikeExpression, LogicalExpression, NaturalIdentifier, NotEmptyExpression, NotExpression, NotNullExpression, NullExpression, PropertyExpression, PropertySubqueryExpression, SimpleExpression, SimpleSubqueryExpression, SizeExpression, SQLCriterion, SubqueryExpression.

4.3. Les restrictions et les expressions

La classe org.hibernate.criterion.Restrictions permet de créer des critères qui sont des conditions permettant de sélectionner les données à retrouver.

La classe Restrictions est une fabrique qui permet de créer des critères de recherche sous la forme d'instance de type Criterion. Les critères proposés encapsulent les opérateurs SQL standards.

Elle propose des méthodes statiques pour créer des instances des différentes implémentations de l'interface Criterion proposées par Hibernate. Ces critères sont utilisés pour définir quels seront les occurrences qui seront retournées par le résultat de la requête.

Exemple :

```
SELECT * FROM Personne WHERE IdPers=1;
```

Exemple :

```
List personnes =
session.createCriteria(Personne.class)
.add(Restrictions.eq("id", 1l))
.list();
```

Les conditions standards de SQL sont encapsulées dans des objets de type Criterion : pour obtenir une de leur instance, il faut utiliser les méthodes statiques de la classe Restrictions dont les principales sont :

Méthode	Rôle
Criterion allEq(Map properties)	Méthode utilitaire qui permet de facilement vérifier que plusieurs propriétés ont une valeur particulière. Les clés du paramètre de type Map correspondent aux noms des propriétés concernées
LogicalExpression and(Criterion lhs, Criterion rhs)	Créer un critère de type "and" qui est vrai si les deux critères sont évalués à vrai
Criterion between(String propertyName, Object lo, Object hi)	Permet d'appliquer une contrainte SQL de type "between" : la valeur de la propriété dont le nom est fourni en paramètre doit être comprise entre les deux valeurs fournies
Conjunction conjunction()	Créer un objet de type Conjunction qui permet d'utiliser un critère de type and simplement en invoquant sa méthode add() pour chaque critère à prendre en compte. Le critère encapsulé dans l'objet de type Conjunction sera true si tous les critères qu'il contient sont true
Disjunction disjunction()	Créer un objet de type Disjunction qui permet d'utiliser un critère de

	type or simplement en invoquant sa méthode add() pour chaque critère à prendre en compte. Le critère encapsulé dans l'objet de type Disjunction sera true si au moins un des critères qu'il contient est true
SimpleExpression eq(String propertyName, Object value)	Permet d'appliquer une contrainte SQL de type égalité : la valeur de la propriété doit être égale à la valeur fournie en paramètre
PropertyExpression eqProperty(String property1, String property2)	La valeur des deux propriétés doit être égale
SimpleExpression ge(String propertyName, Object value)	Permet d'appliquer une contrainte SQL de type "supérieur ou égal" : la valeur de la propriété doit être supérieure ou égale à la valeur fournie en paramètre
PropertyExpression geProperty(String property1, String property2)	La valeur de la propriété doit être supérieure ou égale à la valeur de la seconde propriété fournie en paramètre
SimpleExpression gt(String propertyName, Object value)	Permet d'appliquer une contrainte SQL de type "supérieur à" : la valeur de la propriété doit être supérieure à la valeur fournie en paramètre
PropertyExpression gtProperty(String property1, String property2)	La valeur de la propriété doit être supérieure à la valeur de la seconde propriété fournie en paramètre
Criterion idEq(Object value)	permet d'appliquer une contrainte SQL de type égalité sur l'identifiant : la valeur de la propriété qui est l'identifiant doit être égale à celle fournie
Criterion ilike(String property, Object value)	Joue le même rôle que la méthode like() mais en étant insensible à la casse
Criterion ilike(String property, String value, MatchMode mode)	Joue le même rôle que la méthode like() mais en étant insensible à la casse et sans utiliser la syntaxe de l'opérateur like. MatchMode est une énumération qui peut prendre les valeurs START, END, ANYWHERE, ou EXACT.
Criterion in(String property, Collection values)	La valeur de la propriété dont le nom est fourni en paramètre doit être égale à l'une de celles fournies dans la collection
Criterion in(String propertyName, Collection values)	Permet d'appliquer une contrainte SQL de type "in" : la valeur de la propriété dont le nom est fourni en

	paramètre doit être égale à l'une de celles fournies dans le tableau
Criterion isEmpty(String property)	Le contenu de la collection de la propriété dont le nom est fourni en paramètre ne doit pas avoir d'éléments
Criterion isNotEmpty(String property)	Le contenu de la collection de la propriété dont le nom est fourni en paramètre doit avoir au moins un élément
Criterion isNotNull(String propertyName)	Permet d'appliquer une contrainte SQL de type "is not null" : la valeur de la propriété dont le nom est fourni en paramètre doit être non null
Criterion isNull(String propertyName)	Permet d'appliquer une contrainte SQL de type "is null" : la valeur de la propriété dont le nom est fourni en paramètre doit être null
SimpleExpression le(String property, Object value)	La valeur de la propriété dont le nom est fourni en paramètre doit être inférieur ou égal à la valeur fournie
PropertyExpression leProperty(String property1, String property2)	La valeur de la propriété fournie en paramètre doit être inférieur ou égal à la valeur fournie
SimpleExpression like(String property, Object value)	La valeur de la propriété dont le nom est fourni en paramètre doit respecter le motif de l'opérateur sql like fourni en paramètre
SimpleExpression like(String property, String value, MatchMode mode)	La valeur de la propriété dont le nom est fourni en paramètre doit respecter le motif de l'opérateur sql like déterminé à partir des paramètres valeur et mode. MatchMode est une énumération qui peut prendre les valeurs START, END, ANYWHERE, ou EXACT.
SimpleExpression lt(String property, Object value)	La valeur de la propriété doit être inférieure à la valeur fournie en paramètre
PropertyExpression ltProperty(String property1, String property2)	La valeur de la première propriété doit être inférieure à la seconde
SimpleExpression ne(String propertyName, Object value)	Permet d'appliquer une contrainte SQL de type "est différent de" : la valeur de la propriété dont le nom est fourni en paramètre doit être différente de la valeur fournie
PropertyExpression neProperty(String propertyName, String property2)	La valeur des deux propriétés fournies en paramètres doit être différente

otherPropertyName)	
Criterion not(Criterion expression)	L'évaluation du critère fourni en paramètre doit être false
LogicalExpression or(Criterion lhs, Criterion rhs)	L'évaluation d'un des deux critères fourni en paramètre doit être true
Criterion sqlRestriction(String sql)	Appliquer une restriction en sql natif
Criterion sqlRestriction(String sql, Object[] values, Type[] types)	Appliquer une restriction en sql natif qui va utiliser les paramètres fournis
Criterion sqlRestriction(String sql, Object value, Type type)	Appliquer une restriction en sql natif qui va utiliser le paramètre fourni

Certaines de ces méthodes attendent un paramètre de type Criterion qui permet de faire des combinaisons de critères.

Les opérateurs de comparaison sont encapsulés dans des méthodes de la classe Restrictions : eq(), lt(), le(), gt(), ge().

Exemple :

```
List personnes = session.createCriteria(Personne.class)
.add(Restrictions.lt("dateNais", dateSaisie)).list();
```

La classe Restrictions propose aussi des méthodes pour les opérateurs SQL : like, between, in, is null, is not null ...

Exemple :

```
List personnes = session.createCriteria(Personne.class)
.add(Restrictions.between("dateNais", dateDeb, dateFin))
.add(Restrictions.like("nom", "Dup%")).list();
```

La classe Restrictions propose aussi des méthodes qui permettent de faire des comparaisons entre propriétés.

Exemple :

```
List personnes = session.createCriteria(Personne.class)
.add(Restrictions.eqProperty("nom", "prenom")).list();
```

L'utilisation de la méthode sqlRestriction() peut être très pratique mais elle peut nuire à la portabilité entre bases de données.

Pour préciser l'alias de la table courante dans la portion de requête SQL fournie en paramètre de la méthode sqlRestriction(), il faut utiliser la syntaxe "{alias}".

Il est possible d'utiliser les méthodes or() et and() pour réaliser des combinaisons de critères.

Exemple :

```
List personnes = session.createCriteria(Personne.class)
.add(Restrictions.or(Restrictions.eq("prenom", "Jean"),
Restrictions.eq("prenom", "Paul"))).list();
```

Remarque : il est préférable d'utiliser la classe Restrictions plutôt que sa classe fille Expressions qui est deprecated.

4.4. Les projections et les aggregations

La classe org.hibernate.criterion.Projection permet de préciser un champ qui sera retourné dans le résultat de la requête : ce champ peut être issu d'une table, du calcul d'une aggrégation, de la définition d'un alias, ...

Pour ajouter un champ, il faut passer le nom du champ en paramètre de la méthode statique property() de la classe Projection. L'instance retournée est passée en paramètre de la méthode setProjection().

La classe org.hibernate.criterion.Projections est une fabrique pour créer des instances de type Projection.

Pour préciser plusieurs champs, il faut utiliser la méthode propertyList() de la classe ProjectionList.

Exemple :

```
SELECT NOM, PRENOM FROM PERSONNE
```

Exemple :

```
List resultats = session.createCriteria(Personne.class)
.setProjection(Projections.projectionList()
.add(Projections.property("nom"))
.add(Projections.property("prenom"))).list();
System.out.println("nb personnes= " + resultats.size());
Iterator it = resultats.iterator();
while (it.hasNext()) {
Object[] donnees = (Object[]) it.next();
System.out.println("Nom : " + donnees[0]
+ " Prenom : " + donnees[1]);
}
```

Pour appliquer une collection de type Projection aux critères de la requête, il faut utiliser la méthode setProjection() de la classe Criteria. Une collection d'objets de type Projection est encapsulée dans un objet de type ProjectionList. La méthode add() permet d'ajouter une Projection à la collection.

Exemple :


```
List resultats = session.createCriteria(Personne.class)
.setProjection(Projections.rowCount()).list();
Long valeur = (Long) resultats.get(0);
System.out.println("nb personnes = " + valeur);
```

Les données de certaines requêtes doivent parfois être groupées ou intervenir dans un calcul d'agrégation : il faut pour cela utiliser les fonctionnalités encapsulées dans la classe Projections. Toutes les fonctions d'agrégation de la classe Projections sont des méthodes statiques.

La classe Projections possède plusieurs méthodes statiques :

Méthode	Rôle
static Projections alias(Projection projection, String alias)	Assigner un alias
static AgregateProjection avg(String property)	Calculer la moyenne du champ dont le nom est fourni en paramètre
static CountProjection count(String property)	Calculer le nombre d'occurences du champ dont le nom et fourni en paramètre
static CountProjection countDistinct(String property)	Calculer le nombre d'occurences distinctes du champ dont le nom et fourni en paramètre
static Projection distinct(Projection projection)	Ne retourner que des valeurs uniques (supprimer les valeurs en doublon).
static PropertyProjection groupProperty(String property)	Grouper les résultats sur la propriété fournie
static IdentifierProjection id()	Renvoyer l'identifiant
static AggregateProjection max(String property)	Déterminer la plus grande valeur pour le champ dont le nom est fourni en paramètre
static AggregateProjection min(String property)	Déterminer la plus petite valeur pour le champ dont le nom est fourni en paramètre
static ProjectionList projectionList()	Retourner une collection de projections
static PropertyProjection property(String property)	Ajouter la propriété fournie en paramètre
static Projection rowCount()	Calculer le nombre d'occurences retournées par la requête
static Projection sqlGroupProjection(String sql, String groupBy, String[] columnAliases, Type[] types)	Ajouter du code SQL spécifique à la base de données utilisée pour déterminer un groupage. Le

	paramètre groupBy peut contenir une clause GROUP BY
static Projection sqlProjection(String sql, String[] columnAliases, Type[] types)	Ajouter du code SQL spécifique à la base de données utilisée pour déterminer la liste de champs retournée par la requête
static AggregateProjection sum(String property)	Calculer la somme des valeurs pour le champ dont le nom est fourni en paramètre

Lors de l'utilisation d'opérateurs d'agrégation, il est fréquent de grouper les données par rapport à un champ particulier. La classe Projections possède la méthode groupProperty() qui permet de définir une clause "group by" qui sera utilisée avec le nom du champ fourni en paramètre.

Exemple :

```
SELECT COUNT(ID) FROM PERSONNE GROUP BY
TAILLE
```

Exemple :

```
List resultats = session.createCriteria(Personne.class)
.setProjection(Projections.projectionList()
.add(Projections.count("id"))
.add(Projections.groupProperty("
nom")))
.list();

Iterator it = resultats.iterator();
while (it.hasNext()) {
Object[] donnees = (Object[]) it.next();
System.out.println("Nombre : " + donnees[0] + " taille : " +
donnees[1]);
}
```

5. Le tri des résultats

Il est possible de demander le tri des résultats de la requête en utilisant la méthode addOrderO de la classe Criteria et la classe Order.

La classe org.hibernate.criterion.Order permet d'encapsuler une clause de tri dans la requête en précisant le sens (ascendant ou descendant) sur un champ.

Les méthodes asc() et desc() permettent respectivement de définir un ordre ascendant ou descendant sur les résultats de la requête.

Elle possède plusieurs méthodes :

Méthode	Rôle
static Order asc(String)	permet de demander un tri ascendant sur le nom du champ fourni en paramètre de la méthode
static Order desc(String)	permet de demander un tri descendant sur le nom du champ fourni en paramètre de la méthode
Order ignoreCase()	permet de demander d'ignorer la casse lors du tri des données

Exemple :

```
List resultats =
session.createCriteria(Personne.class)
    .add(Restrictions.between("dateNais", dateDeb, dateFin))
    .addOrder(Order.desc("dateNais"))
    .addOrder(Order.asc("nom"));
```

6. La jointure de tables

Il est très fréquent d'avoir à effectuer une ou plusieurs jointures sur les tables d'une requête pour obtenir les données souhaitées.

En SQL, la jointure se fait en utilisant les tables dans la clause from et en indiquant les conditions de la jointure.

Exemple :

```
SELECT P.*, A.* FROM Personne P, Adresse A WHERE
P.adresse_id=A.id AND A.Nom = »Dupond»;
```

En HQL, il est possible de charger les données d'objets dépendants en utilisant la clause "left join fetch" :

Exemple :

```
from Personne personne where Personne.nom = :nom left join
fetch personne.adresse
```

La méthode setFetchMode() permet de faire une jointure avec l'API Criteria.

Exemple :

```
package com.jmdoudoux.test.hibernate;

import java.util.Iterator;
import java.util.List;
import org.hibernate.FetchMode;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.AnnotationConfiguration;

public class TestHibernateCriteria {
```

```
public static void main(String args[]) {
    SessionFactory sessionFactory = new
AnnotationConfiguration().configure()
        .buildSessionFactory();

    Transaction transaction = null;
    List personnes = null;
    Session session = sessionFactory.openSession();

    try {
        transaction = session.beginTransaction();
        personnes = session.createCriteria(Personne.class)
            .setFetchMode("adresse", FetchMode.JOIN)
            .list();
        System.out.println("nbpersonnes = " + personnes.size());

        Iterator it = personnes.iterator();
        while (it.hasNext()) {
            Personne personne = (Personne) it.next();

            System.out.println("Personne : " + personne);
            System.out.println(" Adresse : " + personne.getAdresse());
        }

        transaction.commit();
    } catch (Exception e) {
        transaction.rollback();
        e.printStackTrace();
    } finally {
        session.close();
    }
    sessionFactory.close();
}
```

La méthode setFetchMode() attend deux paramètres :

- le nom de la classe de l'entité dont la table sera jointe avec celle de l'entité courante
- le mode de récupération des données :
DEFAULT (valeur configuré dans le mapping),
EAGER (deprecated : utiliser JOIN), JOIN (récupération des données par jointure), LAZY (deprecated : utiliser SELECT), SELECT (récupération des données par une requête dédiée)

Parfois, il est nécessaire de joindre les entités mais il est inutile de récupérer les données de l'entité jointe : dans ce cas la jointure n'est utile que pour définir un ou plusieurs critères de la requête

Exemple :

```
from Personne p join p.adresse a where a.cp = '54000'
```

Il est possible d'utiliser la méthode createCriteria() pour créer une jointure :

Exemple :

```
package com.jmdoudoux.test.hibernate;

import java.util.Iterator;
import java.util.List;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.AnnotationConfiguration;
import org.hibernate.criterion.Restrictions;

public class TestHibernateCriteria {

    public static void main(String args[]) {
        SessionFactory sessionFactory = new
AnnotationConfiguration().configure()
().buildSessionFactory

();

        Transaction transaction = null;
        List personnes = null;

        Session session = sessionFactory.openSession();
        try {
            transaction = session.beginTransaction();
            personnes = session.createCriteria(Personne.class)
.createCriteria("adresse", "a")
.add(Restrictions.eq("a.cp", "54700")).list();

            System.out.println("nb personnes = " + personnes.size());

            Iterator it = personnes.iterator();
            while (it.hasNext()) {
                Personne personne = (Personne) it.next();
                System.out.println("Personne : " + personne);
                System.out.println(" Adresse : " +
personne.getAdresse());
            }

            transaction.commit();
        } catch (Exception e) {
            transaction.rollback();
            e.printStackTrace();
        } finally {
            session.close();
        }

        sessionFactory.close();
    }
}
```

Attention : dans ce cas, le premier paramètre de la méthode createCriteria() est le nom d'une propriété et non le nom de classe.

Il est aussi possible d'utiliser la méthode createAlias() qui évite d'avoir à créer une nouvelle instance de la classe Criteria.

Exemple :

```
personnes = session.createCriteria(Personne.class)
.createAlias("adresse", "a")
.add(Restrictions.eq("a.cp", "54000")).list();
```

Les restrictions sont aussi utilisables avec une jointure dans laquelle les données de la table jointe sont récupérées.

Exemple :

```
SELECT G.*, P.* FROM Personnes P, Groupe G
WHERE G.IdGroup=P.IdGroup AND G.IdGroup=1;
```

Exemple :

```
List groupes = session.createCriteria(Groupe.class)
.setFetchMode("Personne", FetchMode.JOIN)
.add(Restrictions.eq("IdGroup", "1")).list();
```

7. La création de critères à partir de données

La classe Example permet de créer des critères par l'exemple : pour cela, il faut instancier une entité et lui affecter les différentes valeurs qui seront utilisées comme critères.

La méthode static create() qui prends en paramètre une instance d'une entité va utiliser l'inspection pour rechercher les propriétés qui possèdent une valeur et générer une instance de type Example contenant les critères correspondants aux propriétés renseignées.

Exemple :

```
transaction = session.beginTransaction();
Date dateDebut = new GregorianCalendar(1980,
Calendar.JANUARY, 01).getTime();
Date dateFin = new GregorianCalendar(1981,
Calendar.JANUARY, 01).getTime();

Personne personneExemple = new Personne();
personneExemple.setNom("Dupond");
personneExemple.setPrenom("Michel");
Example exemple = Example.create(personneExemple).
ignoreCase().excludeZeroes();
List<Personne> resultats =
session.createCriteria(Personne.class).add(exemple)
.add(Restrictions.between("dateNais", dateDebut, dateFin)).list();
System.out.println("nbpersonnes = " + personnes.size());
Iterator it = personnes.iterator();
while (it.hasNext()) {
    Personne personne = (Personne) it.next();
    System.out.println("Personne : " + personne);
    System.out.println(" Adresse : " + personne.getAdresse());
}
transaction.commit();
```

Il est possible de configurer le comportement de la classe Example en utilisant les différentes méthodes qu'elle propose à cette fin :

Méthode	Rôle
Example enableLike()	Utiliser l'opérateur like pour toutes les propriétés de type String

Example enableLike(MatchMode matchMode)	Utiliser l'opérateur like pour toutes les propriétés de type String avec la stratégie de correspondance fournie en paramètre
Example excludeNone()	Ne pas exclure les propriétés dont la valeur est null ou zéro
Example excludeProperty(String name)	Ignorer la propriété dont le nom est fourni en paramètre
Example excludeZeroes()	Ignorer les propriétés dont la valeur est zéro
Example ignoreCase()	Ignorer la casse des propriétés de type String
Example setEscapeCharacter(Character escapeCharacter)	Préciser le caractère d'échappement utilisé dans la clause like

Cette API est particulièrement utile si le nombre de propriétés est important. Comme la classe Example hérite de la classe Criterion, elle peut être utilisée pour créer les critères "simple" et utiliser d'autres classes pour les critères plus complexes ou spécifiques.

8. Le choix entre HQL et l'API Criteria

L'API Criteria est très puissante et elle est particulièrement bien adaptée pour certaines tâches (notamment la création de requête dynamique à la volée impliquant de nombreux critères optionnels comme par exemple dans un formulaire de recherche multi critères).

Dans ces cas, sa mise en oeuvre peut permettre d'avoir un code propre, plus sûr et plus maintenable.

Cependant, son utilisation ne peut pas toujours être généralisée à tous les cas de figure car l'utilisation de HQL est parfois préférable, notamment si la requête n'est pas dynamique.

Il est par exemple préférable d'externaliser les requêtes HQL lorsque cela est possible car cela possède plusieurs avantages :

- faciliter de maintenance
- faciliter pour recenser et optimiser les requêtes
- faciliter pour mettre les requêtes en cache

Il est donc nécessaire de bien choisir entre HQL et l'API Criteria en fonction des besoins et de leur adéquation avec ce que proposent les deux solutions qui se recouvrent mais sont aussi complémentaires.

5. MAPPING DES ASSOCIATIONS VIA XML ET ANNOTATIONS

5.1. Hibernate One To One Annotation et XML Mapping

Etape 1: Création de la relation One-to-one entre 2 tables Mysql qui sont mappées par clé primaire Employee_ID.

```
/* EMPLOYEE table */
CREATE TABLE employee (
  employee_id BIGINT(10) NOT NULL
  AUTO_INCREMENT,
  firstname VARCHAR(50) NULL DEFAULT NULL,
  PRIMARY KEY (employee_id)
)ENGINE=InnoDB;
```

```
/* EMPLOYEEDETAIL table */
CREATE TABLE employeeDetail (
  employee_id BIGINT(20) NOT NULL
  AUTO_INCREMENT,
  country VARCHAR(50) NULL DEFAULT NULL,
  PRIMARY KEY (employee_id),
  CONSTRAINT FKEMPL FOREIGN KEY
  (employee_id) REFERENCES employee
  (employee_id)
)ENGINE=InnoDB;
```

Etape 2. Création ou génération des modèles de classe pour les entités bean en utilisant des annotations jpa

Avec l'outil HibernateTools, on peut générer des classes annotées en cochant l'option "**generate EJB3 annotations**" qui se trouve dans l'onglet "exporter" du wizard de configuration de la génération du code.

Employee.java

```
package exa;
import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.OneToOne;
import javax.persistence.Table;
@Entity @Table(name="EMPLOYEE")
public class Employee {
  @Id @GeneratedValue
  @Column(name="employee_id")
  private Long employeeId;
  @Column(name="firstname")
  private String firstname;
```

```
@OneToOne(mappedBy="employee",
cascade=CascadeType.ALL)
private EmployeeDetail employeeDetail;
public Employee() { }
public Employee(String firstname) {
  this.firstname = firstname;
}
```

```
// Getter and Setter methods
}
```

EmployeeDetail.java

```
package exa;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.OneToOne;
import javax.persistence.PrimaryKeyJoinColumn;
import javax.persistence.Table;

import org.hibernate.annotations.GenericGenerator;
import org.hibernate.annotations.Parameter;

@Entity
@Table(name="EMPLOYEEDETAIL")
public class EmployeeDetail {
    @Id
    @Column(name="employee_id", unique=true,
    nullable=false)
    @GeneratedValue(generator="gen")
    @GenericGenerator(name="gen", strategy="foreign",
    parameters=@Parameter(name="property",
    value="employee"))
    private Long employeeId;

    @Column(name="country")
    private String country;
    @OneToOne
    @PrimaryKeyJoinColumn
    private Employee employee;
    public EmployeeDetail() {}
    public EmployeeDetail( String country) {
        this.country = country;
    }
    // Getter and Setter methods
}
```

Etape 3: Création ou génération des fichiers de mapping XML (HBM)**Employee.hbm.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
.....
<hibernate-mapping package="exa">
    <class name="Employee" table="EMPLOYEE">
        <id name="employeeId"
        column="EMPLOYEE_ID">
            <generator class="native" />
        </id>
        <one-to-one name="employeeDetail"
        class="exa.EmployeeDetail" cascade="save-update">
        </one-to-one>
        <property name="firstname" column="firstname" />
    </class>
</hibernate-mapping>
```

EmployeeDetail.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
.....
<hibernate-mapping package="exa">
```

```
<class name="EmployeeDetail"
table="EMPLOYEEDETAIL">
    <id name="employeeId" type="java.lang.Long">
        <column name="EMPLOYEE_ID" />
        <generator class="foreign">
            <param name="property">employee</param>
        </generator>
    </id>
    <one-to-one name="employee"
    class="exa.Employee"
    constrained="true"></one-to-one>
    <property name="country" column="COUNTRY"/>
</class>
</hibernate-mapping>
```

Création du fichier de configuration (hibernate.cfg.xml)

```
<?xml version='1.0' encoding='utf-8'?>
.....
<hibernate-configuration>
    <session-factory>
        <!-- Database connection settings -->
        <property
        name="connection.driver_class">com.mysql.jdbc.Driver
        </property>
        <property
        name="connection.url">jdbc:mysql://localhost:3306/tutori
        al</property>
        <property
        name="connection.username">root</property>
        <property
        name="connection.password"></property>
        <property
        name="dialect">org.hibernate.dialect.MySQLDialect</pr
        operty>
        <property name="show_sql">true</property>
        <property
        name="hbm2ddl.auto">validate</property>
        <mapping class="exa.EmployeeDetail"/>
        <mapping class="exa.Employee"/>
        <mapping resource="exa/Employee.hbm.xml"/>
        <mapping
        resource="exa/EmployeeDetail.hbm.xml"/>
    </session-factory>
</hibernate-configuration>
```

Classe Main pour tester One-to-one mapping

```
session.beginTransaction();
EmployeeDetail employeeDetail = new
EmployeeDetail( "U.S.");
Employee employee = new Employee("Nina");
employee.setEmployeeDetail(employeeDetail);
employeeDetail.setEmployee(employee);

session.save(employee);

List<Employee> employees =
session.createQuery("from Employee").list();
for (Employee em : employees) {
    System.out.println(emp.getFirstName() + " , "+
    emp.getEmployeeDetail().getCountry());
}
session.getTransaction().commit();
```

5.2. Hibernate One To Many Annotation et XML Mapping

Exemple 1: d'une relation OneToMany et inverse ManyToOne

EMPLOYEE (EMP_ID, FIRSTNAME)

PHONE (ID, NUMBER, #OWNER_ID)

Relation OneToMany et inverse ManyToOne annotations

```
@Entity
public class Employee {
    @Id @Column(name="EMP_ID")
    private long id;
    ...
    @OneToMany(mappedBy="owner")
    private List<Phone> phones;
    ...
}

@Entity
public class Phone {
    @Id private long id;
    ...
    @ManyToOne(fetch=FetchType.LAZY)
    @JoinColumn(name="OWNER_ID")
    // @JoinColumn(name="OWNER_ID", referencedColumn
    // name="EMP_ID")
    private Employee owner;
    ...
}
```

Relation OneToMany et inverse ManyToOne via XML en JPA 2

Employee.hbm.xml

```
<entity name="Employee" class="exa.Employee"
access="FIELD">
    <attributes>
        <id name="id"/>
    </attributes>
    <property name="firstname" type="string"/>
    <one-to-many name="phones" target-
entity="exa.Phone" mapped-by="owner"/>
    </attributes>
</entity>
```

Phone.hbm.xml

```
<entity name="Phone" class="exa.Phone"
access="FIELD">
    <attributes>
        <id name="id"/>
    </attributes>
    <property name="number" type="string"/>
    <many-to-one name="owner" fetch="LAZY">
        <join-column name="OWNER_ID"/>
    </many-to-one>
    <!-- <join-column name="OWNER_ID"
referencedColumnName="EMP_ID"/> -->
    </attributes>
</entity>
```

Exemple 2: OneToMany en utilisant une table de jointure (JoinTable)

EMPLOYEE (EMP_ID, FIRSTNAME)

PHONE (ID, NUMBER)

EMP_PHONE(EMP_ID, PHONE_ID)

OneToMany en utilisant JoinTable annotation

```
@Entity
public class Employee {
    @Id
    @Column(name="EMP_ID")
    private long id;
    ...
    @OneToMany
    @JoinTable( name="EMP_PHONE",
        joinColumns={ @JoinColumn(name="EMP_ID",
        referencedColumnName="EMP_ID") },
        inverseJoinColumns={ @JoinColumn(name="PHONE_I
        D", referencedColumnName="ID", unique=true) }
    )
    private List<Phone> phones;
    ...
}

public class Phone {
    @Id
    @Column(name="ID")
    private long id;
    ....
    // cas bidirectionnel (optionnel)
    @ManyToOne (cascade=CascadeType.ALL)
    private Employee employee ;
    .....
}
```

OneToMany en utilisant JoinTable XML

Employee.hbm.xml

```
<entity name="Employee" class="exa.Employee"
access="FIELD">
    <attributes>
        <id name="id"> <column name="EMP_ID"/>
        </id>
        <one-to-many name="phones">
            <join-table name="EMP_PHONE">
                <join-column name="EMP_ID" referenced-
column-name="EMP_ID"/>
                <inverse-join-
column name="PHONE_ID" referenced-column-
name="ID" unique="true" />
            </join-table>
        </one-to-many>
    </attributes>
</entity>
```

Exemple 3: Hibernate One To Many Mapping Annotations et XML

Étape 1: Création de la relation One-to-Many entre 2 tables Mysql sachant qu'un département comprend plusieurs employés et un employé travaille dans un seul département

```
CREATE TABLE department (
    department_id BIGINT(20) NOT NULL
    AUTO_INCREMENT,
    dept_name VARCHAR(50) NOT NULL DEFAULT '0',
    PRIMARY KEY (department_id)
)
```

```
ENGINE=InnoDB;
CREATE TABLE employee (
  employee_id BIGINT(10) NOT NULL
  AUTO_INCREMENT,
  firstname VARCHAR(50) NULL DEFAULT NULL,
  PRIMARY KEY (employee_id), INDEX FK_DEPT
  (department_id), [ CONSTRAINT FK_DEPT]
  FOREIGN KEY (department_id) REFERENCES
  department (department_id) ON DELETE CASCADE
  ON UPDATE CASCADE )
ENGINE=InnoDB;
```

on ajoute dans la définition de la clé étrangère l'instruction ON DELETE CASCADE ON UPDATE CASCADE de manière à ce qu'une mise à jour ou une suppression dans la table parente soit impactée dans la table enfant

Etape 2. Création ou génération des modèles de classe pour les entités bean sans utiliser des annotations jpa

Avec l'outil HibernateTools, on peut générer des classes non annotées en cochant l'option « Use Java 5 Syntax » qui se trouve dans l'onglet "exporter" du wizard de configuration de la génération du code.

File: Department.java

```
package exa;
import java.util.Set;
public class Department {
  private Long departmentId;
  private String departmentName;
  private Set<Employee> employees;
  // Getter and Setter methods
}
```

File: Employee.java

```
package exa;
public class Employee {
  private Long employeeId;
  private String firstname;
  private Department department;
  public Employee() {
  }

  public Employee(String firstname) {
    this.firstname = firstname;
  }

  // Getter and Setter methods
}
```

Etape 3: Création ou génération des fichiers de mapping XML

File: Employee.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
.....
<hibernate-mapping package="exa">
  <class name="Employee" table="EMPLOYEE">
    <id name="employeeId"
    column="EMPLOYEE_ID">
      <generator class="native" />
```

```
    </id>
    <property name="firstname" column="firstname" />
    <many-to-one name="department"
    class="exa.Department" fetch="select">
      <column name="department_id" not-
      null="true" />
    </many-to-one>
  </class>
</hibernate-mapping>
```

File: Department.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
.....
<hibernate-mapping package="exa">
  <class name="Department" table="DEPARTMENT">
    <id name="departmentId" type="java.lang.Long"
    column="DEPARTMENT_ID">
      <generator class="native" />
    </id>
    <property name="departmentName"
    column="DEPT_NAME"/>
    <set name="employees" table="employee"
    inverse="true" lazy="true" fetch="select">
      <key><column name="department_id" not-
      null="true" />
    </key>
    <one-to-many class="exa.Employee" />
  </set>
</class>
</hibernate-mapping>
```

Création du fichier de configuration (hibernate.cfg.xml)

```
<?xml version='1.0' encoding='utf-8'?>
.....
<hibernate-configuration>
  <session-factory>
    <property name="connection.driver_class">
    com.mysql.jdbc.Driver</property>
    <property
    name="connection.url">jdbc:mysql://localhost/base
    </property>
    <property
    name="connection.username">root</property>
    <property
    name="connection.password"></property>
    <property
    name="dialect">org.hibernate.dialect.MySQLDialect
    </property>
    <property name="hbm2ddl.auto">validate</property>
    <mapping resource="exa/Employee.hbm.xml"/>
    <mapping resource="exa/Departement.hbm.xml"/>
  </session-factory>
</hibernate-configuration>
```

File: Main.java

```
session.beginTransaction();
Department department = new Department();
department.setDepartmentName("Sales");
session.save(department);

Employee emp1 = new Employee("Nina");
Employee emp2 = new Employee("Tony");
```

```
emp1.setDepartment(department);
emp2.setDepartment(department);
```

```
session.save(emp1);
session.save(emp2);
```

```
session.getTransaction().commit();
session.close();
```

Remarque:

Hibernate peut aussi mapper (persister) des collections comme **java.util.Set**, **java.util.Collection**, **java.util.List**, **java.util.Map**, **java.util.SortedSet**, **java.util.SortedMap**.

Tous les mappings de collection, comme **<set>**, **<list>**, **<map>**, **<bag>**, **<array>** ont les propriétés suivantes

name : le nom de la propriété contenant la collection

table (optionnel - par défaut = nom de la propriété) : le nom de la table de la collection (non utilisé pour les associations one-to-many)

lazy (optionnel - par défaut = true) : peut être utilisé pour désactiver l'initialisation tardive et spécifier que l'association est toujours rapportée,

inverse (optionnel - par défaut = false) : définit cette collection comme l'extrémité "inverse" de l'association bidirectionnelle

cascade (optionnel - par défaut = none) : active les opérations de cascade vers les entités filles
sort (optionnel) : spécifie une collection triée

order-by (optionnel, seulement à partir du JDK1.4) : spécifie une colonne de table (ou des colonnes) qui définit l'ordre d'itération de Map, Set ou Bag, avec en option asc ou desc

where (optionnel) : spécifie une condition SQL arbitraire WHERE à utiliser au chargement ou à la suppression d'une collection (utile si la collection ne doit contenir qu'un sous ensemble des données disponibles)

fetch (optionnel, par défaut = select) : à choisir entre récupération par jointures externes, récupération par selects séquentiels, et récupération par sous-selects séquentiels

Les clefs étrangères d'une collection

Les instances d'une collection sont distinguées dans la base par la clef étrangère de l'entité qui possède la collection. Cette clef étrangère est référencée comme la(es) *colonne(s) de la clef de la collection* de la table de la collection. La colonne de la clef de la collection est mappée par l'élément **<key>**.

Il peut y avoir une contrainte de nullité sur la colonne de la clef étrangère. Pour les associations unidirectionnelles un vers plusieurs, la colonne de la clef étrangère peut être nulle par défaut, donc vous pourriez avoir besoin de spécifier not-null="true".

```
class name="Department">
  <id name="depid" column="dep_id"/>
  <set name="employees">
    <key column="dep_id" not-null="true"/>
    <one-to-many class="employee"/>
  </set>
</key column="departementid" not-null="true"/>
```

La contrainte de la clef étrangère peut utiliser ON DELETE CASCADE.

```
<key column="dep_id" on-delete="cascade"/>
```

Tous les mappings de collection, exceptés ceux avec les sémantiques d'ensemble (set) et de sac (bag), ont besoin d'une **colonne d'index** dans la table de la collection - une colonne qui mappe un index.

Si votre table n'a pas de colonne d'index, et que vous souhaitez tout de même utiliser **List** comme type de propriété, vous devriez mapper la propriété comme un **<bag>** Hibernate. Un sac (bag) ne garde pas son ordre quand il est récupéré de la base de données, mais il peut être optionnellement trié ou ordonné.

Pour implementer Bag dans le mapping one-to-many mapping , utiliser **java.util.List** correspondant à **<bag>**.

File: Department.java modifié
 package exa;

```
import java.util.ArrayList;
import java.util.List;

public class Department {

    private Long departmentId;

    private String departmentName;

    private List<Employee>
employees;

    // Getter and Setter methods

}
```

File: Department.hbm.xml modifié

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://hibernate.sourceforge.net/hibernate-
mapping-3.0.dtd">

<hibernate-mapping package="exa">

    <class name="Department" table="DEPARTMENT">

        <id name="departmentId" type="java.lang.Long"
            column="DEPARTMENT_ID" >
            <generator class="native" />
        </id>
```



```

<property name="departmentName"
  column="DEPT_NAME"/>

<bag name="employees" table="employee"
  inverse="true" lazy="true" fetch="select">
  <key>
    <column name="department_id" not-null="true" />
  </key>
  <one-to-many class="exa.Employee" />
</bag>

</class>
</hibernate-mapping>

```

5.3. Hibernate Many To Many Annotation et XML Mapping

Exemple 1: ManyToMany en utilisant une table de jointure (JoinTable)

EMPLOYEE (EMP_ID, FIRSTNAME)

PROJET (ID, NAME)

EMP_PROJ (EMP_ID, PROJ_ID)

Relation ManyToMany avec Jointable annotations

```

@Entity
public class Employee {
    @Id
    @Column(name="ID")
    private long id;
    ...
    @ManyToMany
    @JoinTable(
        name="EMP_PROJ",
        joinColumns=@JoinColumn(name="EMP_ID",
            referencedColumnName="ID"),
        inverseJoinColumns=@JoinColumn(name="PROJ_ID",
            referencedColumnName="ID"))
    private List<Project> projects;
    ....
}

public class Project {
    @Id
    // @GeneratedValue
    @Column(name="ID")
    private int id;
    private String name;

    @ManyToMany(mappedBy="projects",
        cascade = CascadeType.ALL)
    private List<Employee> employees;
    ....
}

```

Relation XML ManyToMany avec jointable

Employee.hbm.xml

```

<entity name="Employee" class="exa.Employee"
  access="FIELD">
  <attributes>
    <id name="id">

```

```

      <column name="EMP_ID"/>
    </id>
    <set name="projects" table="EMP_PROJ"
      lazy="true" cascade="none" sort="natural" optimistic-
      lock="false">
      <key column="EMP_ID" not-
      null="true" />
      <many-to-many class="exa.Project"
      column="PROJ_ID" />
    </set>
  </attributes>
</entity>

```

File: Project.hbm.xml

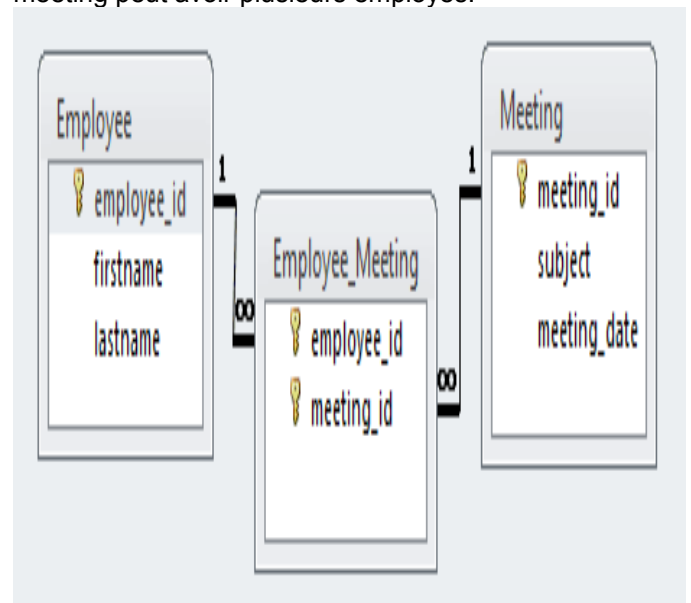
```

<?xml version="1.0" encoding="UTF-8"?>
.....
<hibernate-mapping package="exa">
  <class name="Project" table="Project">
    <id name="id" type="java.lang.Long" column="ID">
      <generator class="native" />
    </id>
    <property name="name" column="name" />
    <set name="employees" table="EMP_PROJ"
      inverse="true" lazy="true" fetch="select">
      <key column="EMPLOYEE_ID" />
      <many-to-many column="EMP_ID"
      class="Employee" />
    </set>
  </class>
</hibernate-mapping>

```

Exemple 2: Hibernate Many To Many Annotation et XML Mapping

Etape 1: Création de la relation ManyToMany entre 2 tables Mysql Employee et Meeting sachant que chaque employé peut assister à plusieurs meetings et chaque meeting peut avoir plusieurs employés.



```

CREATE TABLE `employee` (
  `employee_id` BIGINT(10) NOT NULL
  AUTO_INCREMENT,
  `firstname` VARCHAR(50) NULL DEFAULT NULL,
  PRIMARY KEY (`employee_id`)) ENGINE = InnoDB;

```

```
CREATE TABLE `meeting` (
  `meeting_id` BIGINT(20) NOT NULL
  AUTO_INCREMENT,
  `subject` VARCHAR(50) NOT NULL,
  PRIMARY KEY (`meeting_id`)) ENGINE = InnoDB;

CREATE TABLE `employee_meeting` (
  `employee_id` BIGINT(20) NOT NULL,
  `meeting_id` BIGINT(20) NOT NULL,
  PRIMARY KEY (`employee_id`, `meeting_id`),
  INDEX `FK_MEETING` (`meeting_id`),
  CONSTRAINT `FK_EMPLOYEE` FOREIGN KEY
  (`employee_id`) REFERENCES `employee`
  (`employee_id`), CONSTRAINT `FK_MEETING`
  FOREIGN KEY (`meeting_id`) REFERENCES `meeting`
  (`meeting_id`)) ENGINE = InnoDB;
```

Etape 2. Création ou génération des modèles de classe pour les entités bean en utilisant des annotations jpa

File: Employee.java
package exa;

```
import java.util.HashSet;
import java.util.Set;
import javax.persistence.CascadeType;
import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.Id;
import javax.persistence.JoinColumn;
import javax.persistence.JoinTable;
import javax.persistence.ManyToMany;
import javax.persistence.Table;
@Entity
@Table(name="EMPLOYEE")
public class Employee {
    @Id
    @Column(name="EMPLOYEE_ID")
    @GeneratedValue
    private Long employeeId;
    @Column(name="FIRSTNAME")
    private String firstname;
    @ManyToMany(cascade = {CascadeType.ALL})
    @JoinTable(name="EMPLOYEE_MEETING",
        joinColumns={@JoinColumn(name="EMPL
        OYEE_ID")},
        inverseJoinColumns={@JoinColumn(name
        ="MEETING_ID")})
    private Set<Meeting> meetings = new
    HashSet<Meeting>();

    public Employee() {}

    public Employee(String firstname, String lastname) {
        this.firstname = firstname;
    }

    // Getter and Setter methods
}
```

File: Meeting.java

```
package exa;
import java.util.HashSet;
```

```
import java.util.Set;
import javax.persistence.*;
@Entity
@Table(name="MEETING")
public class Meeting {
    @Id
    @Column(name="MEETING_ID")
    @GeneratedValue
    private Long meetingId;
    @Column(name="SUBJECT")
    private String subject;
    @ManyToMany(mappedBy="meetings")
    private Set<Employee> employees = new
    HashSet<Employee>();
    public Meeting(String subject) {this.subject = subject;}

    // Getter and Setter methods
}
```

Etape 3. Création des fichiers de mapping XML (hbm)

File: Employee.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
.....
<hibernate-mapping package="exa">
    <class name="Employee" table="EMPLOYEE">
        <id name="employeeId"
        column="EMPLOYEE_ID">
            <generator class="native" />
        </id>
        <property name="firstname" />
        <set name="meetings"
        table="EMPLOYEE_MEETING"
        inverse="false" lazy="true" fetch="select"
        cascade="all">
            <key column="EMPLOYEE_ID" />
            <many-to-many column="MEETING_ID"
            class="Meeting" />
        </set>
        </class>
    </hibernate-mapping>
```

File: Meeting.hbm.xml

```
<?xml version="1.0" encoding="UTF-8"?>
.....
<hibernate-mapping package="exa">
    <class name="Meeting" table="MEETING">
        <id name="meetingId" type="java.lang.Long"
        column="MEETING_ID">
            <generator class="native" />
        </id>
        <property name="subject" column="SUBJECT" />
        <set name="employees"
        table="EMPLOYEE_MEETING"
        inverse="true" lazy="true" fetch="select">
            <key column="EMPLOYEE_ID" />
            <many-to-many column="MEETING_ID"
            class="Employee" />
        </set>
        </class>
    </hibernate-mapping>
```

Création du fichier de configuration (hibernate.cfg.xml)

```
<?xml version='1.0' encoding='utf-8'?>
.....
<hibernate-configuration>
  <session-factory>
    <property>
      name="connection.driver_class">com.mysql.jdbc.Driver
    </property>
    <property>
      name="connection.url">jdbc:mysql://localhost:3306/base
    </property>
    <property>
      name="connection.username">root</property>
    <property>
      name="connection.password"></property>
    <property>
      name="dialect">org.hibernate.dialect.MySQLDialect</pr
      operty>
    <property name="hbm2ddl.auto">validate</property>

    <mapping class="exa.Meeting"/>
    <mapping class="exa.Employee"/>
    <mapping resource="exa/Employee.hbm.xml"/>
    <mapping resource="exa/Meeting.hbm.xml"/>

  </session-factory>
</hibernate-configuration>
```

File: Main.java

```
session.beginTransaction();

Meeting meeting1 = new Meeting("meeting 1");
Meeting meeting2 = new Meeting("meeting 2");

Employee employee1 = new Employee("Sergey");
Employee employee2 = new Employee("Larry",);

employee1.getMeetings().add(meeting1);
employee1.getMeetings().add(meeting2);
employee2.getMeetings().add(meeting1);

session.save(employee1);
session.save(employee2);
session.getTransaction().commit();
session.close();
```

6. Complément sur le mapping XML des associations

Nous illustrerons tous nos exemples avec les classes Person et Address.

6.1. Association unidirectionnelle

6.1.1. plusieurs à un

Une *association plusieurs-à-un (many-to-one) unidirectionnelle* est le type que l'on rencontre le plus souvent dans les associations unidirectionnelles.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="address"
    column="addressId"
    not-null="true"/>
</class>
```

```
<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
</class>
```

```
create table Person ( personId bigint not null primary
key, addressId bigint not null )
create table Address ( addressId bigint not null primary
key )
```

6.1.2. un à un

une *association un-à-un (one-to-one) sur une clé étrangère* est presque identique. La seule différence est sur la contrainte d'unicité que l'on impose à cette colonne.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="address"
    column="addressId"
    unique="true"
    not-null="true"/>
</class>
```

```
<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
</class>
create table Person ( personId bigint not null primary
key, addressId bigint not null unique )
create table Address ( addressId bigint not null primary
key )
```

Une *association un-à-un (one-to-one) unidirectionnelle sur une clé primaire* utilise un générateur d'identifiant particulier. (Remarquez que nous avons inversé le sens de cette association dans cet exemple.)

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
</class>

<class name="Address">
  <id name="id" column="personId">
    <generator class="foreign">
      <param name="property">person</param>
    </generator>
  </id>
  <one-to-one name="person" constrained="true"/>
</class>
create table Person ( personId bigint not null primary key
)
create table Address ( personId bigint not null primary
key )
```

6.1.3. un à plusieurs

Une *association un-à-plusieurs (one-to-many) unidirectionnelle sur une clé étrangère* est vraiment inhabituelle, et n'est pas vraiment recommandée.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <set name="addresses">
    <key column="personId"
      not-null="true"/>
    <one-to-many class="Address"/>
  </set>
</class>
```

```
<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
</class>
create table Person ( personId bigint not null primary key
)
create table Address ( addressId bigint not null primary
key, personId bigint not null )
```

Nous pensons qu'il est préférable d'utiliser une table de jointure pour ce type d'association.

6.2. Associations unidirectionnelles avec tables de jointure

6.2.1. un à plusieurs

Une *association unidirectionnelle un-à-plusieurs (one-to-many) avec une table de jointure* est un bien meilleur

choix. Remarquez qu'en spécifiant `unique="true"`, on a changé la multiplicité plusieurs-à-plusieurs (many-to-many) pour un-à-plusieurs (one-to-many).

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <set name="addresses" table="PersonAddress">
    <key column="personId"/>
    <many-to-many column="addressId"
      unique="true"
      class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
</class>
create table Person ( personId bigint not null primary key
)
create table PersonAddress ( personId not null,
addressId bigint not null primary key )
create table Address ( addressId bigint not null primary
key )
```

6.2.2. plusieurs à un

Une *association plusieurs-à-un (many-to-one) unidirectionnelle sur une table de jointure* est très fréquente quand l'association est optionnelle.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <join table="PersonAddress"
    optional="true">
    <key column="personId" unique="true"/>
    <many-to-one name="address"
      column="addressId"
      not-null="true"/>
  </join>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
</class>
create table Person ( personId bigint not null primary key
)
create table PersonAddress ( personId bigint not null
primary key, addressId bigint not null )
create table Address ( addressId bigint not null primary
key )
```

6.2.3. un à un

Une *association unidirectionnelle un-à-un (one-to-one) sur une table de jointure* est extrêmement rare mais envisageable.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <join table="PersonAddress"
    optional="true">
    <key column="personId"
      unique="true"/>
    <many-to-one name="address"
      column="addressId"
      not-null="true"
      unique="true"/>
  </join>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
</class>
create table Person ( personId bigint not null primary key
)
create table PersonAddress ( personId bigint not null
primary key, addressId bigint not null unique )
create table Address ( addressId bigint not null primary
key )
```

6.2.4. plusieurs à plusieurs

Finalement, nous avons l'*association unidirectionnelle plusieurs-à-plusieurs (many-to-many)*.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <set name="addresses" table="PersonAddress">
    <key column="personId"/>
    <many-to-many column="addressId"
      class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
</class>
create table Person ( personId bigint not null primary key
)
create table PersonAddress ( personId bigint not null,
addressId bigint not null, primary key (personId,
addressId) )
create table Address ( addressId bigint not null primary
key )
```

6.3. Associations bidirectionnelles

6.3.1. un à plusieurs / plusieurs à un

Une *association bidirectionnelle plusieurs à un (many-to-one)* est le type d'association que l'on rencontre le plus souvent. (c'est la façon standard de créer des relations parents/enfants.)

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="address"
    column="addressId"
    not-null="true"/>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
  <set name="peersons" inverse="true">
    <key column="addressId"/>
    <one-to-many class="Person"/>
  </set>
</class>
create table Person ( personId bigint not null primary
key, addressId bigint not null )
create table Address ( addressId bigint not null primary
key )
```

Si vous utilisez une List (ou toute autre collection indexée) vous devez paramétrer la colonne key de la clé étrangère à not null, et laisser Hibernate gérer l'association depuis l'extrémité collection pour maintenir l'index de chaque élément (rendant l'autre extrémité virtuellement inverse en paramétrant update="false" et insert="false"):

```
<class name="Person">
  <id name="id"/>
  ...
  <many-to-one name="address"
    column="addressId"
    not-null="true"
    insert="false"
    update="false"/>
</class>

<class name="Address">
  <id name="id"/>
  ...
  <list name="people">
    <key column="addressId" not-null="true"/>
    <list-index column="peopleIdx"/>
    <one-to-many class="Person"/>
  </list>
</class>
```

6.3.2. Un à un

Une *association bidirectionnelle un à un (one-to-one)* sur une clé étrangère est aussi très fréquente.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <many-to-one name="address"
    column="addressId"
    unique="true"
    not-null="true"/>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
  <one-to-one name="person"
    property-ref="address"/>
</class>
create table Person ( personId bigint not null primary
key, addressId bigint not null unique )
create table Address ( addressId bigint not null primary
key )
```

Une *association bidirectionnelle un-à-un (one-to-one)* sur une clé primaire utilise un générateur particulier d'id.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <one-to-one name="address"/>
</class>

<class name="Address">
  <id name="id" column="personId">
    <generator class="foreign">
      <param name="property">person</param>
    </generator>
  </id>
  <one-to-one name="person"
    constrained="true"/>
</class>
create table Person ( personId bigint not null primary key
)
create table Address ( personId bigint not null primary
key )
```

6.4. Associations bidirectionnelles avec table de jointure

6.4.1. un à plusieurs / plusieurs à un

Une *association bidirectionnelle un-à-plusieurs (one-to-many)* sur une table de jointure. Remarquez que inverse="true" peut s'appliquer sur les deux extrémités de l'association, sur la collection, ou sur la jointure.

```
<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <set name="addresses"
    table="PersonAddress">
```

```

    <key column="personId"/>
    <many-to-many column="addressId"
      unique="true"
      class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
  <join table="PersonAddress"
    inverse="true"
    optional="true">
    <key column="addressId"/>
    <many-to-one name="person"
      column="personId"
      not-null="true"/>
  </join>
</class>
create table Person ( personId bigint not null primary key
)
create table PersonAddress ( personId bigint not null,
addressId bigint not null primary key )
create table Address ( addressId bigint not null primary
key )

```

6.4.2. Un à un

Une *association bidirectionnelle un-à-un (one-to-one)* sur une table de jointure est extrêmement rare mais envisageable.

```

<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <join table="PersonAddress"
    optional="true">
    <key column="personId"
      unique="true"/>
    <many-to-one name="address"
      column="addressId"
      not-null="true"
      unique="true"/>
  </join>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
  <join table="PersonAddress"
    optional="true"
    inverse="true">
    <key column="addressId"
      unique="true"/>
    <many-to-one name="person"
      column="personId"
      not-null="true"
      unique="true"/>
  </join>
</class>

```

```

create table Person ( personId bigint not null primary key
)
create table PersonAddress ( personId bigint not null
primary key, addressId bigint not null unique )
create table Address ( addressId bigint not null primary
key )

```

6.4.3. plusieurs à plusieurs

Finallement nous avons l'*association bidirectionnelle plusieurs à plusieurs*.

```

<class name="Person">
  <id name="id" column="personId">
    <generator class="native"/>
  </id>
  <set name="addresses" table="PersonAddress">
    <key column="personId"/>
    <many-to-many column="addressId"
      class="Address"/>
  </set>
</class>

<class name="Address">
  <id name="id" column="addressId">
    <generator class="native"/>
  </id>
  <set name="people" inverse="true"
    table="PersonAddress">
    <key column="addressId"/>
    <many-to-many column="personId"
      class="Person"/>
  </set>
</class>
create table Person ( personId bigint not null primary key
)
create table PersonAddress ( personId bigint not null,
addressId bigint not null, primary key (personId,
addressId) )
create table Address ( addressId bigint not null primary
key )

```